

Chương 9 – Học không giám sát

Cuốn sổ tay này chứa tất cả mã mẫu và giải pháp cho các bài tập trong chương 9.

 [Open in Colab](#)  [Open in Kaggle](#)

Thiết lập

Dự án này yêu cầu Python 3.7 trở lên.

```
import sys

assert sys.version_info >= (3, 7)
```

Nó cũng yêu cầu Scikit-Learn $\geq 1.0.1$.

```
from packaging import version
import sklearn

assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

Như chúng ta đã làm ở các chương trước, hãy định nghĩa kích thước phông chữ mặc định để làm cho các hình ảnh đẹp hơn:

```
import matplotlib.pyplot as plt

plt.rc('font', size=14)
plt.rc('axes', labelsiz=14, titlesize=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsiz=10)
plt.rc('ytick', labelsiz=10)
```

Và hãy tạo thư mục `images/unsupervised_learning` (nếu nó chưa tồn tại), và định nghĩa hàm `save_fig()` mà sẽ được sử dụng trong toàn bộ cuốn sổ này để lưu các hình ảnh với độ phân giải cao cho cuốn sách:

```
from pathlib import Path

IMAGES_PATH = Path() / "images" / "unsupervised_learning"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

Đầu tiên, hãy nhập một vài mô-đun phổ biến, đảm bảo rằng Matplotlib vẽ hình ảnh ngay trong dòng và chuẩn bị một hàm để lưu các hình ảnh.

Phân nhóm

Giới thiệu – Phân loại so với Nhóm

```
# extra code - this cell generates and saves Figure 9-1

import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

data = load_iris()
X = data.data
y = data.target
```

```

data.target_names

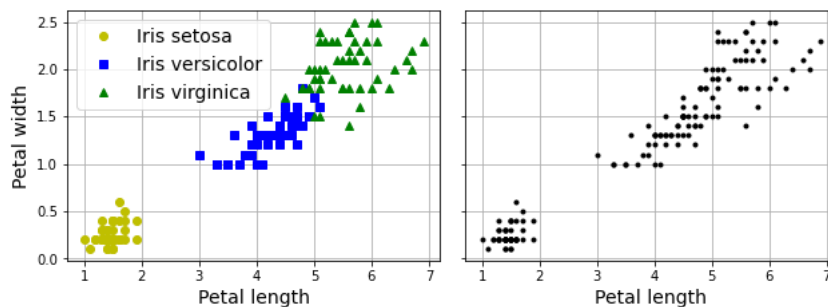
plt.figure(figsize=(9, 3.5))

plt.subplot(121)
plt.plot(X[y==0, 2], X[y==0, 3], "yo", label="Iris setosa")
plt.plot(X[y==1, 2], X[y==1, 3], "bs", label="Iris versicolor")
plt.plot(X[y==2, 2], X[y==2, 3], "g^", label="Iris virginica")
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.grid()
plt.legend()

plt.subplot(122)
plt.scatter(X[:, 2], X[:, 3], c="k", marker=".")
plt.xlabel("Petal length")
plt.tick_params(labelleft=False)
plt.gca().set_axisbelow(True)
plt.grid()

save_fig("classification_vs_clustering_plot")
plt.show()

```



Lưu ý: ô tiếp theo cho thấy cách **mô hình hỗn hợp Gaussian** (được giải thích sau trong chương này) thực sự có thể phân tách các cụm này khá tốt bằng cách sử dụng cả 4 đặc trưng: chiều dài & chiều rộng cánh hoa, và chiều dài & chiều rộng đài hoa. Mã này ánh xạ mỗi cụm thành một lớp. Thay vì mã hóa cứng ánh xạ, mã chọn lớp phổ biến nhất cho mỗi cụm bằng cách sử dụng hàm `scipy.stats.mode()`:

```

# extra code

import numpy as np
from scipy import stats
from sklearn.mixture import GaussianMixture

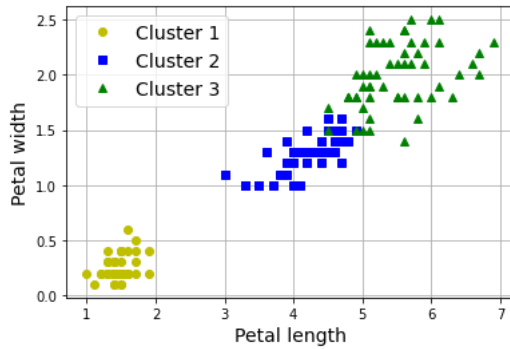
y_pred = GaussianMixture(n_components=3, random_state=42).fit(X).predict(X)

mapping = {}
for class_id in np.unique(y):
    mode, _ = stats.mode(y_pred[y==class_id])
    mapping[mode] = class_id

y_pred = np.array([mapping[cluster_id] for cluster_id in y_pred])

plt.plot(X[y_pred==0, 2], X[y_pred==0, 3], "yo", label="Cluster 1")
plt.plot(X[y_pred==1, 2], X[y_pred==1, 3], "bs", label="Cluster 2")
plt.plot(X[y_pred==2, 2], X[y_pred==2, 3], "g^", label="Cluster 3")
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.legend(loc="upper left")
plt.grid()
plt.show()

```



Tỷ lệ số cây iris mà chúng ta đã gán vào cụm đúng là bao nhiêu?

```
(y_pred==y).sum() / len(y_pred)
```

```
0.9666666666666667
```

▼ K-Means

Huấn luyện và dự đoán

Hãy huấn luyện một **bộ phân cụm K-Trung bình** (K-Means clusterer) trên một tập dữ liệu gồm các **khối** (blobs). Nó sẽ cố gắng tìm ra **tâm** của mỗi khối và gán mỗi **thể hiện** (instance) vào khối gần nhất:

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# extra code - the exact arguments of make_blobs() are not important
blob_centers = np.array([[ 0.2,  2.3], [-1.5,  2.3], [-2.8,  1.8],
                          [-2.8,  2.8], [-2.8,  1.3]])
blob_std = np.array([0.4, 0.3, 0.1, 0.1, 0.1])
X, y = make_blobs(n_samples=2000, centers=blob_centers, cluster_std=blob_std,
                  random_state=7)

k = 5
kmeans = KMeans(n_clusters=k, n_init=10, random_state=42)
y_pred = kmeans.fit_predict(X)
```

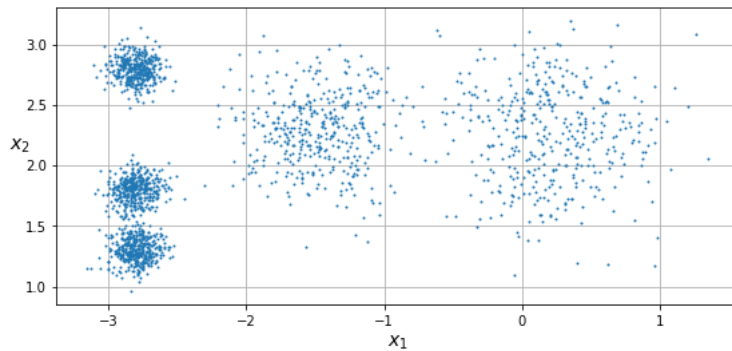
Lưu ý: Xuyên suốt notebook này, khi tham số `n_init` không được đặt khi tạo một bộ ước lượng `KMeans`, tôi đã **tường minh** (explicitly) đặt nó thành `n_init=10` để tránh một cảnh báo về việc giá trị mặc định cho siêu tham số này sẽ thay đổi từ 10 thành `"auto"` trong Scikit-Learn phiên bản 1.4.

Bây giờ chúng ta hãy vẽ chúng:

```
# extra code - this cell generates and saves Figure 9-2

def plot_clusters(X, y=None):
    plt.scatter(X[:, 0], X[:, 1], c=y, s=1)
    plt.xlabel("$x_1$")
    plt.ylabel("$x_2$", rotation=0)

plt.figure(figsize=(8, 4))
plot_clusters(X)
plt.gca().set_axisbelow(True)
plt.grid()
save_fig("blobs_plot")
plt.show()
```



Mỗi thể hiện (instance) được gán vào một trong 5 cụm:

```
y_pred
```

```
array([4, 0, 1, ..., 2, 1, 0], dtype=int32)
```

```
y_pred is kmeans.labels_
```

```
True
```

Và 5 tâm cụm (centroid) (tức là trung tâm cụm) sau đây đã được ước tính:

```
kmeans.cluster_centers_
```

```
array([[ -2.80389616,  1.80117999],
       [  0.20876306,  2.25551336],
       [ -2.79290307,  2.79641063],
       [ -1.46679593,  2.28585348],
       [ -2.80037642,  1.30082566]])
```

Lưu ý rằng thể hiện (KMeans instance) **bảo toàn nhãn** (labels) của các thể hiện mà nó được huấn luyện. Hơi khó hiểu, nhưng trong ngữ cảnh này, **nhãn** (label) của một thể hiện là **chỉ số** của cụm mà thể hiện đó được gán vào (chúng không phải là mục tiêu *targets*, mà là các dự đoán *predictions*):

```
kmeans.labels_
```

```
array([4, 0, 1, ..., 2, 1, 0], dtype=int32)
```

Tất nhiên, chúng ta có thể dự đoán các nhãn của các thể hiện mới:

```
import numpy as np
```

```
X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])
kmeans.predict(X_new)
```

```
array([1, 1, 2, 2], dtype=int32)
```

Ranh giới Quyết định

Hãy vẽ các **ranh giới quyết định** của mô hình. Điều này cho chúng ta một **sơ đồ Voronoi** (Voronoi diagram):

```
# extra code - this cell generates and saves Figure 9-3
```

```
def plot_data(X):
    plt.plot(X[:, 0], X[:, 1], 'k.', markersize=2)
```

```
def plot_centroids(centroids, weights=None, circle_color='w', cross_color='k'):
    if weights is not None:
        centroids = centroids[weights > weights.max() / 10]
    plt.scatter(centroids[:, 0], centroids[:, 1],
                marker='o', s=35, linewidths=8,
                color=circle_color, zorder=10, alpha=0.9)
    plt.scatter(centroids[:, 0], centroids[:, 1],
```

```

marker='x', s=2, linewidths=12,
color=cross_color, zorder=11, alpha=1)

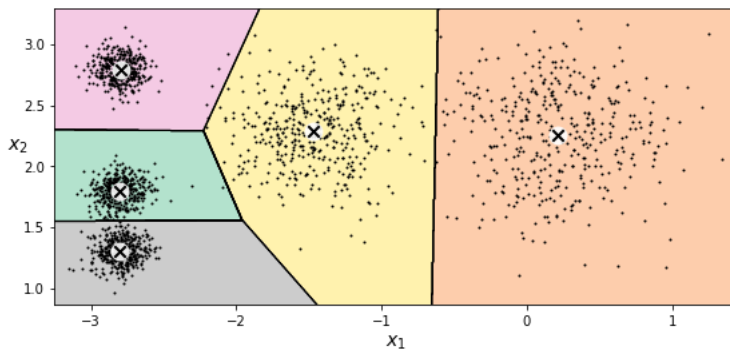
def plot_decision_boundaries(clusterer, X, resolution=1000, show_centroids=True,
                             show_xlabels=True, show_ylabels=True):
    mins = X.min(axis=0) - 0.1
    maxs = X.max(axis=0) + 0.1
    xx, yy = np.meshgrid(np.linspace(mins[0], maxs[0], resolution),
                          np.linspace(mins[1], maxs[1], resolution))
    Z = clusterer.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(Z, extent=(mins[0], maxs[0], mins[1], maxs[1]),
                  cmap="Pastel2")
    plt.contour(Z, extent=(mins[0], maxs[0], mins[1], maxs[1]),
                linewidths=1, colors='k')
    plot_data(X)
    if show_centroids:
        plot_centroids(clusterer.cluster_centers_)

    if show_xlabels:
        plt.xlabel("$x_1$")
    else:
        plt.tick_params(labelbottom=False)
    if show_ylabels:
        plt.ylabel("$x_2$", rotation=0)
    else:
        plt.tick_params(labelleft=False)

plt.figure(figsize=(8, 4))
plot_decision_boundaries(kmeans, X)
save_fig("voronoi_plot")
plt.show()

```



Không tồi! Một số thể hiện gần các cạnh có lẽ đã bị gán nhầm cụm, nhưng nhìn chung thì nó khá tốt.

Phân cụm Cứng (Hard Clustering) so với Phân cụm Mềm (Soft Clustering)

Thay vì chọn một cách tùy tiện cụm gần nhất cho mỗi thể hiện—điều được gọi là **phân cụm cứng** (*hard clustering*)—sẽ tốt hơn nếu đo khoảng cách của mỗi thể hiện đến tất cả 5 **tâm cụm** (centroids). Đây là điều mà phương thức `transform()` thực hiện:

```

kmeans.transform(X_new).round(2)

array([[2.81, 0.33, 2.9 , 1.49, 2.89],
       [5.81, 2.8 , 5.85, 4.48, 5.84],
       [1.21, 3.29, 0.29, 1.69, 1.71],
       [0.73, 3.22, 0.36, 1.55, 1.22]])

```

Bạn có thể xác minh rằng đây thực sự là khoảng cách Euclide giữa mỗi thể hiện và mỗi tâm cụm:

```

# extra code
np.linalg.norm(np.tile(X_new, (1, k)).reshape(-1, k, 2)
               - kmeans.cluster_centers_, axis=2).round(2)

array([[2.81, 0.33, 2.9 , 1.49, 2.89],
       [5.81, 2.8 , 5.85, 4.48, 5.84],
       [1.21, 3.29, 0.29, 1.69, 1.71],
       [0.73, 3.22, 0.36, 1.55, 1.22]])

```

▼ Thuật toán K-Means

Thuật toán **K-Trung bình** (K-Means) là một trong những thuật toán phân cụm nhanh nhất và cũng là một trong những thuật toán đơn giản nhất:

- Đầu tiên, khởi tạo k tâm cụm (centroids) một cách ngẫu nhiên: ví dụ, k thể hiện khác nhau được chọn ngẫu nhiên từ tập dữ liệu và các tâm cụm được đặt tại vị trí của chúng.
- Lặp lại cho đến khi hội tụ (tức là cho đến khi các tâm cụm ngừng di chuyển): * Gán mỗi thể hiện cho tâm cụm gần nhất. * Cập nhật các tâm cụm bằng cách lấy **giá trị trung bình** của các thể hiện được gán cho chúng.

Lớp `KMeans` sử dụng kỹ thuật **khởi tạo được tối ưu hóa** theo mặc định. Để có được thuật toán **K-Trung bình** (K-Means) gốc (chỉ cho mục đích giáo dục), bạn phải đặt `init="random"` và `n_init=1`. Thêm về vấn đề này sẽ có ở phần sau trong chương này.

Hãy chạy thuật toán **K-Trung bình** (K-Means) với 1, 2 và 3 lần **lặp** (iterations), để xem các **tâm cụm** (centroids) di chuyển như thế nào:

```
# extra code - this cell generates and saves Figure 9-4

kmeans_iter1 = KMeans(n_clusters=5, init="random", n_init=1, max_iter=1,
                      random_state=5)
kmeans_iter2 = KMeans(n_clusters=5, init="random", n_init=1, max_iter=2,
                      random_state=5)
kmeans_iter3 = KMeans(n_clusters=5, init="random", n_init=1, max_iter=3,
                      random_state=5)

kmeans_iter1.fit(X)
kmeans_iter2.fit(X)
kmeans_iter3.fit(X)

plt.figure(figsize=(10, 8))

plt.subplot(321)
plot_data(X)
plot_centroids(kmeans_iter1.cluster_centers_, circle_color='r', cross_color='w')
plt.ylabel("$x_2$", rotation=0)
plt.tick_params(labelbottom=False)
plt.title("Update the centroids (initially randomly)")

plt.subplot(322)
plot_decision_boundaries(kmeans_iter1, X, show_xlabels=False,
                        show_ylabels=False)
plt.title("Label the instances")

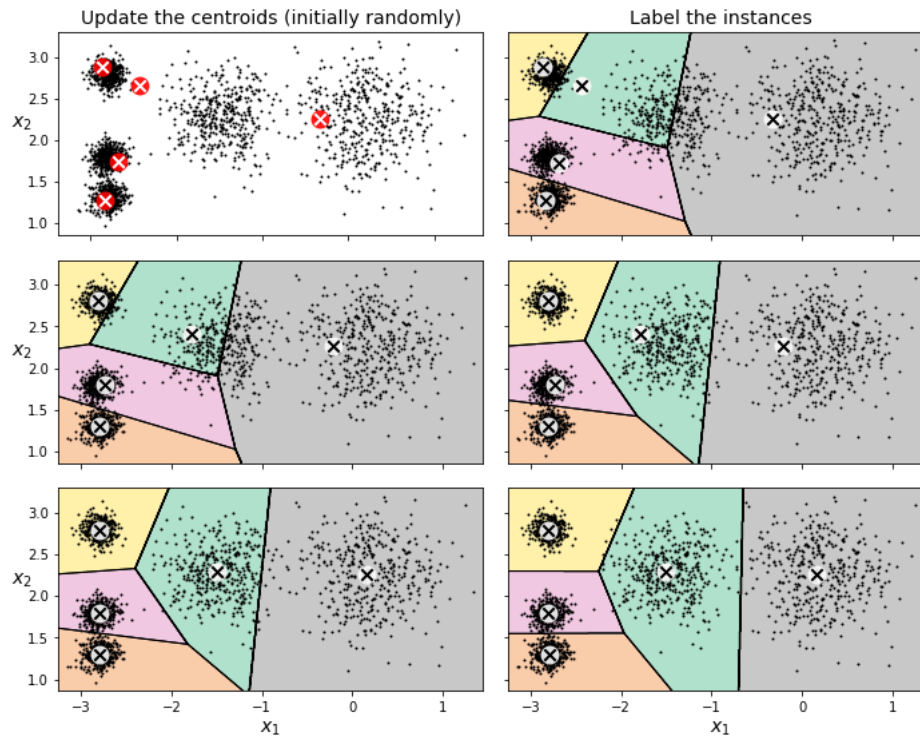
plt.subplot(323)
plot_decision_boundaries(kmeans_iter1, X, show_centroids=False,
                        show_xlabels=False)
plot_centroids(kmeans_iter2.cluster_centers_)

plt.subplot(324)
plot_decision_boundaries(kmeans_iter2, X, show_xlabels=False,
                        show_ylabels=False)

plt.subplot(325)
plot_decision_boundaries(kmeans_iter2, X, show_centroids=False)
plot_centroids(kmeans_iter3.cluster_centers_)

plt.subplot(326)
plot_decision_boundaries(kmeans_iter3, X, show_ylabels=False)

save_fig("kmeans_algorithm_plot")
plt.show()
```



Tính biến động của K-Means

Trong thuật toán **K-Trung bình** (K-Means) gốc, các **tâm cụm** (centroids) chỉ được **khởi tạo ngẫu nhiên**, và thuật toán chỉ chạy một lần **lặp** (iteration) duy nhất để dần dần cải thiện các tâm cụm, như chúng ta đã thấy ở trên.

Tuy nhiên, một vấn đề lớn với phương pháp này là nếu bạn chạy K-Means nhiều lần (hoặc với các **hạt giống ngẫu nhiên** *random seeds* khác nhau), nó có thể hội tụ đến các giải pháp **rất khác nhau**, như bạn có thể thấy dưới đây:

```
# extra code - this cell generates and saves Figure 9-5

def plot_clusterer_comparison(clusterer1, clusterer2, X, title1=None,
                              title2=None):
    clusterer1.fit(X)
    clusterer2.fit(X)

    plt.figure(figsize=(10, 3.2))

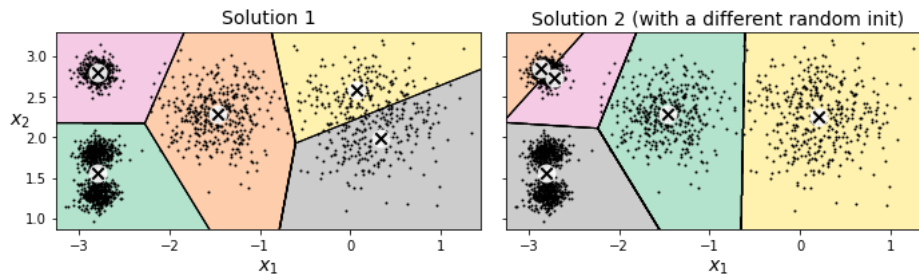
    plt.subplot(121)
    plot_decision_boundaries(clusterer1, X)
    if title1:
        plt.title(title1)

    plt.subplot(122)
    plot_decision_boundaries(clusterer2, X, show_ylabels=False)
    if title2:
        plt.title(title2)

kmeans_rnd_init1 = KMeans(n_clusters=5, init="random", n_init=1, random_state=2)
kmeans_rnd_init2 = KMeans(n_clusters=5, init="random", n_init=1, random_state=9)

plot_clusterer_comparison(kmeans_rnd_init1, kmeans_rnd_init2, X,
                          "Solution 1",
                          "Solution 2 (with a different random init)")

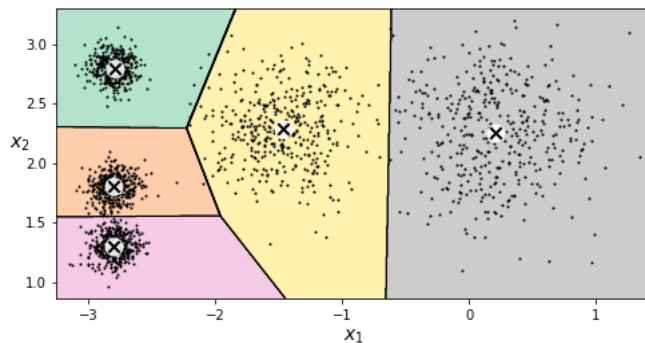
save_fig("kmeans_variability_plot")
plt.show()
```



```
good_init = np.array([[ -3,  3], [ -3,  2], [ -3,  1], [ -1,  2], [  0,  2]])
kmeans = KMeans(n_clusters=5, init=good_init, n_init=1, random_state=42)
kmeans.fit(X)
```

```
KMeans(init=array([[ -3,  3],
                  [ -3,  2],
                  [ -3,  1],
                  [ -1,  2],
                  [  0,  2]]),
       n_clusters=5, n_init=1, random_state=42)
```

```
# extra code
plt.figure(figsize=(8, 4))
plot_decision_boundaries(kmeans, X)
```



✎ Quán tính

Để chọn **mô hình tốt nhất**, chúng ta sẽ cần một cách để đánh giá hiệu suất của mô hình **K-Trung bình** (K-Means). Thật không may, phân cụm là một nhiệm vụ **không giám sát** (unsupervised task), vì vậy chúng ta **không có các mục tiêu** (*targets*). Nhưng ít nhất, chúng ta có thể đo khoảng cách giữa mỗi thể hiện và **tâm cụm** (centroid) của nó. Đây chính là ý tưởng đằng sau số liệu **quán tính** (*inertia*):

```
kmeans.inertia_

211.59853725816836
```

```
kmeans_rnd_init1.inertia_ # extra code

219.43539442771404
```

```
kmeans_rnd_init2.inertia_ # extra code

258.64724728961636
```

Như bạn có thể dễ dàng xác minh, **quán tính** (*inertia*) là **tổng bình phương khoảng cách** giữa mỗi **thể hiện huấn luyện** (training instance) và **tâm cụm** (centroid) gần nhất của nó:

```
# extra code
X_dist = kmeans.transform(X)
(X_dist[np.arange(len(X_dist)), kmeans.labels_] ** 2).sum()

211.59853725816856
```


Phương thức `score()` trả về **quán tính** (*inertia*) **âm**. Tại sao lại là âm? À, đó là vì phương thức `score()` của một **bộ dự đoán** (*predictor*) luôn phải tuân thủ quy tắc "**càng lớn càng tốt**" (*greater is better*).

```
kmeans.score(X)

-211.5985372581684
```

✎ Khởi tạo Đa lần

Vì vậy, một phương pháp để giải quyết vấn đề **tính biến thiên** (variability) là đơn giản là chạy thuật toán **K-Trung bình** (K-Means) nhiều lần với các **khởi tạo ngẫu nhiên** khác nhau, và chọn giải pháp giúp **giảm thiểu quán tính** (*inertia*) nhất.

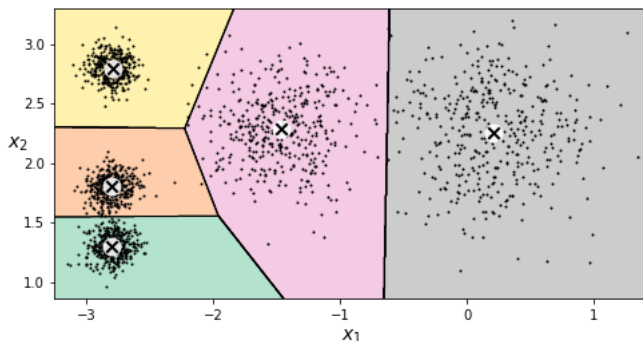
Khi bạn đặt siêu tham số `n_init`, Scikit-Learn sẽ chạy thuật toán gốc `n_init` lần, và chọn giải pháp giúp **giảm thiểu quán tính** (*inertia*) nhất. Theo mặc định, Scikit-Learn đặt `n_init=10`.

```
# extra code
kmeans_rnd_10_inits = KMeans(n_clusters=5, init="random", n_init=10,
                             random_state=2)
kmeans_rnd_10_inits.fit(X)

KMeans(init='random', n_clusters=5, random_state=2)
```

Như bạn có thể thấy, chúng ta kết thúc với **mô hình ban đầu**, vốn chắc chắn là **giải pháp K-Trung bình (K-Means) tối ưu** (ít nhất là về mặt **quán tính** (*inertia*), và giả định $k = 5$).

```
# extra code
plt.figure(figsize=(8, 4))
plot_decision_boundaries(kmeans_rnd_10_inits, X)
plt.show()
```



```
kmeans_rnd_10_inits.inertia_

211.5985372581684
```

✎ Các Phương pháp Khởi tạo Tâm cụm

Thay vì khởi tạo các **tâm cụm** (centroids) hoàn toàn ngẫu nhiên, ta nên ưu tiên khởi tạo chúng bằng thuật toán sau, được đề xuất trong một [bài báo năm 2006](#) bởi David Arthur và Sergei Vassilvitskii:

- Lấy một tâm cụm c_1 , được chọn **đồng nhất** (uniformly) ngẫu nhiên từ tập dữ liệu.
- Lấy một tâm mới c_i , chọn một **thể hiện** $\mathbf{x}_i$ với xác suất: $D(\mathbf{x}_i)^2 / \sum_{j=1}^m D(\mathbf{x}_j)^2$ trong đó $D(\mathbf{x}_i)$ là khoảng cách giữa thể hiện $\mathbf{x}_i$ và tâm cụm gần nhất đã được chọn. Phân bố xác suất này đảm bảo rằng các thể hiện **càng xa** các tâm cụm đã chọn thì càng có nhiều khả năng được chọn làm tâm cụm.
- Lặp lại bước trước cho đến khi tất cả k tâm cụm đã được chọn.

The rest of the K-Means++ algorithm is just regular K-Means. With this initialization, the K-Means algorithm is much less likely to converge to a suboptimal solution, so it is possible to reduce `n_init` considerably. Most of the time, this largely compensates for the additional

complexity of the initialization process.

Để đặt phương pháp khởi tạo là **K-Means++**, chỉ cần đặt `init="k-means++"` (đây thực ra là giá trị mặc định):

✓ K-Trung bình Tăng tốc

Thuật toán **K-Trung bình** (K-Means) đôi khi có thể được **tăng tốc** bằng cách tránh nhiều phép tính khoảng cách không cần thiết: điều này đạt được bằng cách khai thác **bất đẳng thức tam giác** (cho ba điểm A, B và C, khoảng cách AC luôn thỏa mãn $AC \leq AB + BC$) và bằng cách theo dõi các **cận dưới và cận trên** (lower and upper bounds) cho khoảng cách giữa các thể hiện (instances) và tâm cụm (centroids) (xem [bài báo năm 2003](#) này của Charles Elkan để biết thêm chi tiết).

For Elkan's variant of K-Means, use `algorithm="elkan"`. For regular KMeans, use `algorithm="full"`. The default is `"auto"`, which uses the full algorithm since Scikit-Learn 1.1 (it used Elkan's algorithm before that).

✓ Mini-Batch K-Means

Scikit-Learn cũng triển khai một **biến thể** của thuật toán **K-Trung bình** (K-Means) hỗ trợ **mini-batch** (xem [bài báo này](#)):

```
from sklearn.cluster import MiniBatchKMeans

minibatch_kmeans = MiniBatchKMeans(n_clusters=5, n_init=3, random_state=42)
minibatch_kmeans.fit(X)

MiniBatchKMeans(n_clusters=5, random_state=42)
```

Lưu ý: Xuyên suốt notebook này, khi tham số `n_init` không được đặt khi tạo một bộ ước lượng `MiniBatchKMeans`, tôi đã **tường minh** (explicitly) đặt nó thành `n_init=3` để tránh một cảnh báo về việc giá trị mặc định cho siêu tham số này sẽ thay đổi từ 3 thành `"auto"` trong Scikit-Learn phiên bản 1.4.

```
minibatch_kmeans.inertia_

211.65239850433204
```

Sử dụng `MiniBatchKMeans` **cùng với** `mmap` (không có trong sách)

Nếu tập dữ liệu không vừa với bộ nhớ (memory), tùy chọn đơn giản nhất là sử dụng lớp `mmap`, giống như chúng ta đã làm với **PCA tăng dần** (*incremental PCA*) trong chương trước. Đầu tiên, hãy tải **MNIST**:

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False, parser="auto")
```

Hãy phân tách tập dữ liệu:

```
X_train, y_train = mnist.data[:60000], mnist.target[:60000]
X_test, y_test = mnist.data[60000:], mnist.target[60000:]
```

Tiếp theo, hãy ghi tập dữ liệu huấn luyện vào một `mmap`:

```
filename = "my_mnist.mmap"
X_memmap = np.memmap(filename, dtype='float32', mode='write',
                     shape=X_train.shape)
X_memmap[:] = X_train
X_memmap.flush()
```

```
from sklearn.cluster import MiniBatchKMeans

minibatch_kmeans = MiniBatchKMeans(n_clusters=10, batch_size=10,
                                   n_init=3, random_state=42)

minibatch_kmeans.fit(X_memmap)
```

```
MiniBatchKMeans(batch_size=10, n_clusters=10, random_state=42)
```

Hãy vẽ **tỷ lệ quán tính** (*inertia ratio*) và **tỷ lệ thời gian huấn luyện** (*training time ratio*) giữa K-Trung bình Mini-batch và K-Trung bình thông thường:

```
# extra code - this cell generates and saves Figure 9-6

from timeit import timeit

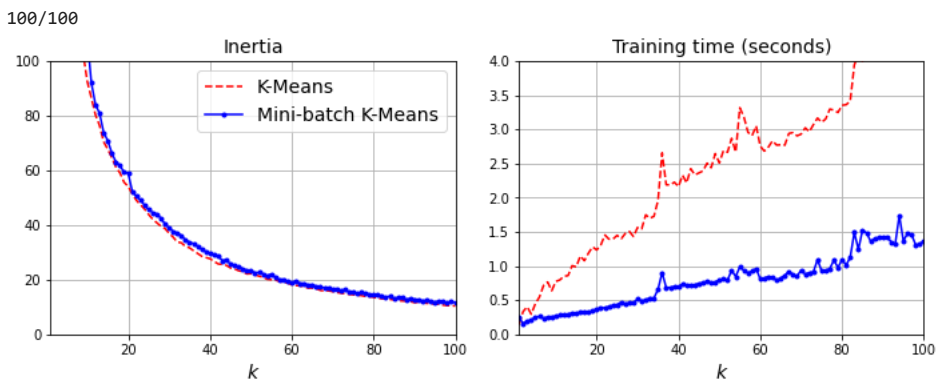
max_k = 100
times = np.empty((max_k, 2))
inertias = np.empty((max_k, 2))
for k in range(1, max_k + 1):
    kmeans_ = KMeans(n_clusters=k, algorithm="lloyd", n_init=10, random_state=42)
    minibatch_kmeans = MiniBatchKMeans(n_clusters=k, n_init=10, random_state=42)
    print(f"\r{k}/{max_k}", end="") # \r returns to the start of line
    times[k - 1, 0] = timeit("kmeans_.fit(X)", number=10, globals=globals())
    times[k - 1, 1] = timeit("minibatch_kmeans.fit(X)", number=10,
                             globals=globals())
    inertias[k - 1, 0] = kmeans_.inertia_
    inertias[k - 1, 1] = minibatch_kmeans.inertia_

plt.figure(figsize=(10, 4))

plt.subplot(121)
plt.plot(range(1, max_k + 1), inertias[:, 0], "r--", label="K-Means")
plt.plot(range(1, max_k + 1), inertias[:, 1], "b.-", label="Mini-batch K-Means")
plt.xlabel("$k$")
plt.title("Inertia")
plt.legend()
plt.axis([1, max_k, 0, 100])
plt.grid()

plt.subplot(122)
plt.plot(range(1, max_k + 1), times[:, 0], "r--", label="K-Means")
plt.plot(range(1, max_k + 1), times[:, 1], "b.-", label="Mini-batch K-Means")
plt.xlabel("$k$")
plt.title("Training time (seconds)")
plt.axis([1, max_k, 0, 4])
plt.grid()

save_fig("minibatch_kmeans_vs_kmeans_plot")
plt.show()
```



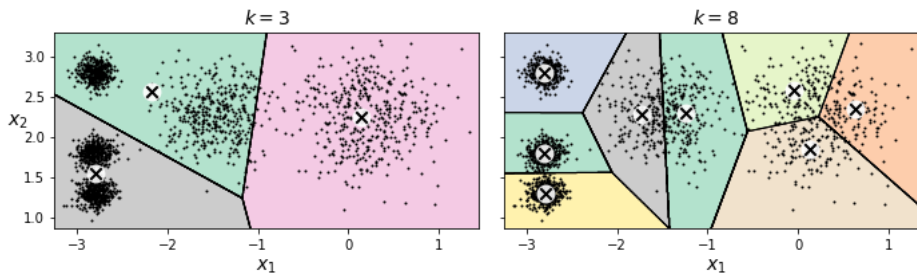
▼ Tìm số lượng cụm tối ưu

Điều gì sẽ xảy ra nếu số lượng cụm được đặt thành giá trị nhỏ hơn hoặc lớn hơn 5?

extra code - this cell generates and saves Figure 9-7

```
kmeans_k3 = KMeans(n_clusters=3, n_init=10, random_state=42)
kmeans_k8 = KMeans(n_clusters=8, n_init=10, random_state=42)

plot_clusterer_comparison(kmeans_k3, kmeans_k8, X, "$k=3$", "$k=8$")
save_fig("bad_n_clusters_plot")
plt.show()
```



Ồi, hai mô hình này trông không ổn lắm. Vậy còn **quán tính** (*inertias*) của chúng thì sao?

kmeans_k3.inertia_

653.2167190021554

kmeans_k8.inertia_

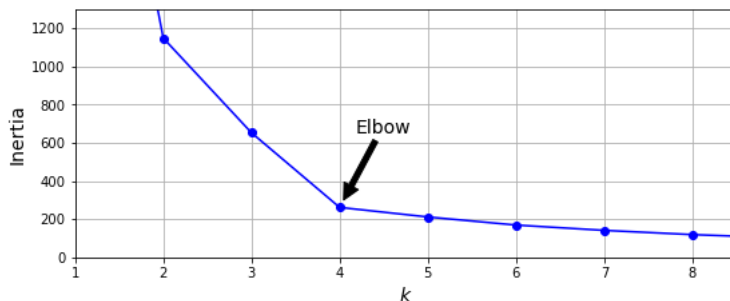
119.11983416102882

Không, chúng ta không thể đơn giản lấy giá trị k giúp **giảm thiểu quán tính** (*inertia*) nhất, vì nó sẽ **tiếp tục giảm** khi chúng ta tăng k . Thật vậy, càng có nhiều cụm, mỗi **thể hiện** (instance) sẽ càng gần **tâm cụm** (centroid) gần nhất của nó, và do đó quán tính sẽ càng thấp. Tuy nhiên, chúng ta có thể vẽ **quán tính** theo hàm của k và phân tích đường cong thu được:

extra code - this cell generates and saves Figure 9-8

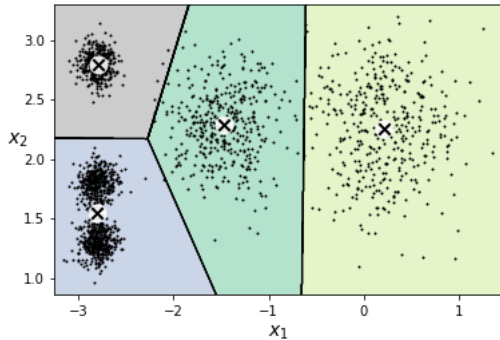
```
kmeans_per_k = [KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
                 for k in range(1, 10)]
inertias = [model.inertia_ for model in kmeans_per_k]

plt.figure(figsize=(8, 3.5))
plt.plot(range(1, 10), inertias, "bo-")
plt.xlabel("$k$")
plt.ylabel("Inertia")
plt.annotate("", xy=(4, inertias[3]), xytext=(4.45, 650),
             arrowprops=dict(facecolor='black', shrink=0.1))
plt.text(4.5, 650, "Elbow", horizontalalignment="center")
plt.axis([1, 8.5, 0, 1300])
plt.grid()
save_fig("inertia_vs_k_plot")
plt.show()
```



Như bạn có thể thấy, có một **cùi chỏ** (*elbow*) tại $k = 4$, điều này có nghĩa là số lượng cụm ít hơn sẽ không tốt, và nhiều cụm hơn sẽ không giúp ích nhiều và có thể cắt đôi các cụm. Vì vậy, $k = 4$ là một lựa chọn khá tốt. Dĩ nhiên, trong ví dụ này nó không hoàn hảo vì nó có nghĩa là hai **khối** (*blobs*) ở phía dưới bên trái sẽ chỉ được coi là một cụm duy nhất, nhưng đây dù sao cũng là một kết quả phân cụm khá tốt.

```
# extra code
plot_decision_boundaries(kmeans_per_k[4 - 1], X)
plt.show()
```



Một phương pháp khác là xem xét **điểm số hình bóng** (*silhouette score*), là **hệ số hình bóng** (*silhouette coefficient*) trung bình trên tất cả các thể hiện. Hệ số hình bóng của một thể hiện bằng $(b - a) / \max(a, b)$ trong đó a là **khoảng cách trung bình** đến các thể hiện khác trong cùng một cụm (nó là **khoảng cách nội cụm trung bình** - *mean intra-cluster distance*), và b là **khoảng cách trung bình đến cụm gần nhất** (*mean nearest-cluster distance*), tức là khoảng cách trung bình đến các thể hiện của cụm gần nhất tiếp theo (được xác định là cụm làm giảm thiểu b , loại trừ cụm của chính thể hiện đó). Hệ số hình bóng có thể thay đổi từ **-1 đến +1**: hệ số gần **+1** có nghĩa là thể hiện đó nằm rõ ràng bên trong cụm của nó và **xa** các cụm khác, trong khi hệ số gần **0** có nghĩa là nó nằm **gần ranh giới cụm**, và cuối cùng, hệ số gần **-1** có nghĩa là thể hiện đó có thể đã bị gán **nhầm cụm**.

Hãy vẽ **điểm số hình bóng** (*silhouette score*) theo hàm của k :

```
from sklearn.metrics import silhouette_score
```

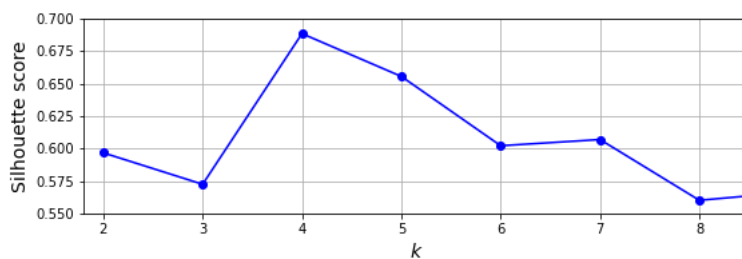
```
silhouette_score(X, kmeans.labels_)
```

```
0.655517642572828
```

```
# extra code - this cell generates and saves Figure 9-9
```

```
silhouette_scores = [silhouette_score(X, model.labels_)
                      for model in kmeans_per_k[1:]]
```

```
plt.figure(figsize=(8, 3))
plt.plot(range(2, 10), silhouette_scores, "bo-")
plt.xlabel("$k$")
plt.ylabel("Silhouette score")
plt.axis([1.8, 8.5, 0.55, 0.7])
plt.grid()
save_fig("silhouette_score_vs_k_plot")
plt.show()
```



Như bạn có thể thấy, hình ảnh trực quan này **phong phú hơn** nhiều so với hình ảnh trước: đặc biệt, mặc dù nó xác nhận rằng $k = 4$ là một lựa chọn rất tốt, nhưng nó cũng nhấn mạnh một thực tế là $k = 5$ cũng **khá tốt**.

Một hình ảnh trực quan thậm chí còn cung cấp nhiều thông tin hơn là khi bạn vẽ **hệ số hình bóng** (*silhouette coefficient*) của mọi thể hiện, được sắp xếp theo cụm mà chúng được gán và theo giá trị của hệ số đó. Đây được gọi là **sơ đồ hình bóng** (*silhouette diagram*):

```
# extra code - this cell generates and saves Figure 9-10
```

```

from sklearn.metrics import silhouette_samples
from matplotlib.ticker import FixedLocator, FixedFormatter

plt.figure(figsize=(11, 9))

for k in (3, 4, 5, 6):
    plt.subplot(2, 2, k - 2)

    y_pred = kmeans_per_k[k - 1].labels_
    silhouette_coefficients = silhouette_samples(X, y_pred)

    padding = len(X) // 30
    pos = padding
    ticks = []
    for i in range(k):
        coeffs = silhouette_coefficients[y_pred == i]
        coeffs.sort()

        color = plt.cm.Spectral(i / k)
        plt.fill_betweenx(np.arange(pos, pos + len(coeffs)), 0, coeffs,
                          facecolor=color, edgecolor=color, alpha=0.7)
        ticks.append(pos + len(coeffs) // 2)
        pos += len(coeffs) + padding

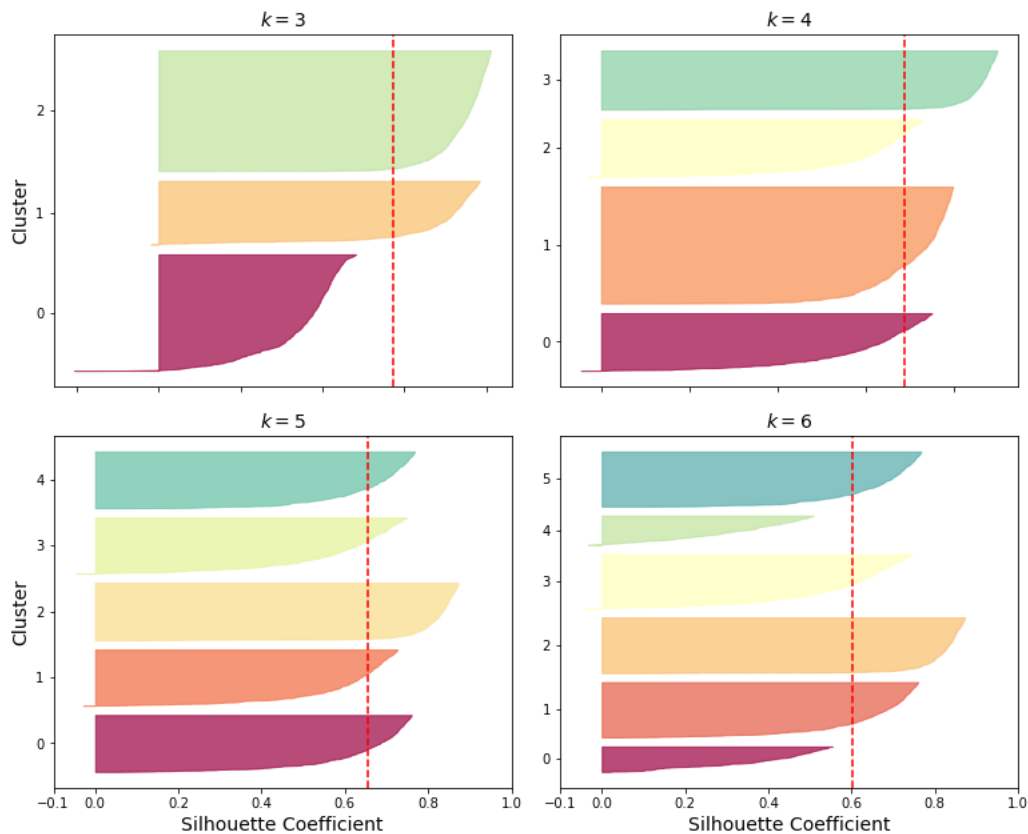
    plt.gca().yaxis.set_major_locator(FixedLocator(ticks))
    plt.gca().yaxis.set_major_formatter(FixedFormatter(range(k)))
    if k in (3, 5):
        plt.ylabel("Cluster")

    if k in (5, 6):
        plt.gca().set_xticks([-0.1, 0, 0.2, 0.4, 0.6, 0.8, 1])
        plt.xlabel("Silhouette Coefficient")
    else:
        plt.tick_params(labelbottom=False)

    plt.axvline(x=silhouette_scores[k - 2], color="red", linestyle="--")
    plt.title(f"$k={k}$")

save_fig("silhouette_analysis_plot")
plt.show()

```



Như bạn có thể thấy, $k = 5$ trông có vẻ là **lựa chọn tốt nhất** ở đây, vì tất cả các cụm đều có **kích thước gần như nhau**, và tất cả chúng đều **vượt qua đường gạch ngang**, đại diện cho **điểm số hình bóng trung bình** (*mean silhouette score*).

▼ Những hạn chế của K-Means

Hãy tạo một tập dữ liệu **khó hơn**, với các **khối kéo dài** (*elongated blobs*) và **mật độ khác nhau** (*varying densities*), và cho thấy rằng **K-Trung bình** (K-Means) gặp khó khăn khi phân cụm nó một cách chính xác:

```
# extra code - this cell generates and saves Figure 9-11

X1, y1 = make_blobs(n_samples=1000, centers=((4, -4), (0, 0)), random_state=42)
X1 = X1.dot(np.array([[0.374, 0.95], [0.732, 0.598]]))
X2, y2 = make_blobs(n_samples=250, centers=1, random_state=42)
X2 = X2 + [6, -8]
X = np.r_[X1, X2]
y = np.r_[y1, y2]

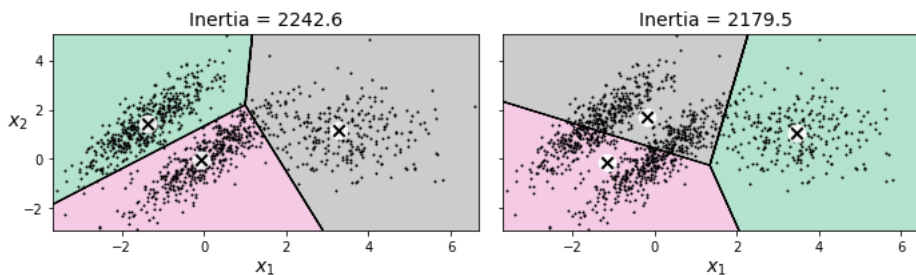
kmeans_good = KMeans(n_clusters=3,
                     init=np.array([[ -1.5, 2.5], [0.5, 0], [4, 0]]),
                     n_init=1, random_state=42)
kmeans_bad = KMeans(n_clusters=3, n_init=10, random_state=42)
kmeans_good.fit(X)
kmeans_bad.fit(X)

plt.figure(figsize=(10, 3.2))

plt.subplot(121)
plot_decision_boundaries(kmeans_good, X)
plt.title(f"Inertia = {kmeans_good.inertia_:.1f}")

plt.subplot(122)
plot_decision_boundaries(kmeans_bad, X, show_ylabels=False)
plt.title(f"Inertia = {kmeans_bad.inertia_:.1f}")

save_fig("bad_kmeans_plot")
plt.show()
```



▼ Sử dụng Phân cụm cho Phân đoạn Ảnh

Tải xuống hình ảnh bộ rùa:

```
# extra code - downloads the ladybug image

import urllib.request

homl3_root = "https://github.com/ageron/handson-ml3/raw/main/"
filename = "ladybug.png"
filepath = IMAGES_PATH / filename
if not filepath.is_file():
    print("Downloading", filename)
    url = f"{homl3_root}/images/unsupervised_learning/{filename}"
    urllib.request.urlretrieve(url, filepath)
```

```
import PIL
```

```
image = np.asarray(PIL.Image.open(filepath))
image.shape
```

```
(533, 800, 3)
```

```
X = image.reshape(-1, 3)
kmeans = KMeans(n_clusters=8, n_init=10, random_state=42).fit(X)
segmented_img = kmeans.cluster_centers_[kmeans.labels_]
segmented_img = segmented_img.reshape(image.shape)
```

extra code - this cell generates and saves Figure 9-12

```
segmented_imgs = []
n_colors = (10, 8, 6, 4, 2)
for n_clusters in n_colors:
    kmeans = KMeans(n_clusters=n_clusters, n_init=10, random_state=42).fit(X)
    segmented_img = kmeans.cluster_centers_[kmeans.labels_]
    segmented_imgs.append(segmented_img.reshape(image.shape))

plt.figure(figsize=(10, 5))
plt.subplots_adjust(wspace=0.05, hspace=0.1)

plt.subplot(2, 3, 1)
plt.imshow(image)
plt.title("Original image")
plt.axis('off')

for idx, n_clusters in enumerate(n_colors):
    plt.subplot(2, 3, 2 + idx)
    plt.imshow(segmented_imgs[idx] / 255)
    plt.title(f"{n_clusters} colors")
    plt.axis('off')

save_fig('image_segmentation_plot', tight_layout=False)
plt.show()
```



▼ Sử dụng Phân cụm cho Học Bán giám sát

Một trường hợp sử dụng khác cho **phân cụm** (*clustering*) là **học bán giám sát** (*semi-supervised learning*), khi chúng ta có **rất nhiều thể hiện không được gán nhãn** (*unlabeled instances*) và **rất ít thể hiện được gán nhãn** (*labeled instances*).

Hãy giải quyết **tập dữ liệu chữ số** (*digits dataset*) vốn là một tập dữ liệu đơn giản **giống MNIST** chứa 1.797 hình ảnh **mức xám** (*grayscale*) **8x8** đại diện cho các chữ số từ 0 đến 9.

```
from sklearn.datasets import load_digits

X_digits, y_digits = load_digits(return_X_y=True)
X_train, y_train = X_digits[:1400], y_digits[:1400]
X_test, y_test = X_digits[1400:], y_digits[1400:]
```

Hãy xem xét **hiệu suất** của một **mô hình hồi quy logistic** (*logistic regression model*) khi chúng ta chỉ có **50 thể hiện được gán nhãn** (*labeled instances*):


```
from sklearn.linear_model import LogisticRegression

n_labeled = 50
log_reg = LogisticRegression(max_iter=10_000)
log_reg.fit(X_train[:n_labeled], y_train[:n_labeled])
```

```
LogisticRegression(max_iter=10000)
```

```
log_reg.score(X_test, y_test)
```

```
0.7481108312342569
```

```
# extra code - measure the accuracy when we use the whole training set
log_reg_full = LogisticRegression(max_iter=10_000)
log_reg_full.fit(X_train, y_train)
log_reg_full.score(X_test, y_test)
```

```
0.906801007556675
```

Tất nhiên, con số này **thấp hơn** nhiều so với lúc trước. Hãy xem làm thế nào chúng ta có thể làm tốt hơn. Đầu tiên, hãy **phân cụm** (*cluster*) tập huấn luyện thành **50 cụm**, sau đó đối với mỗi cụm, hãy tìm **hình ảnh gần tâm cụm nhất**. Chúng ta sẽ gọi những hình ảnh này là **hình ảnh đại diện** (*representative images*):

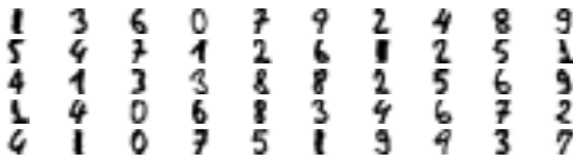
```
k = 50
kmeans = KMeans(n_clusters=k, n_init=10, random_state=42)
X_digits_dist = kmeans.fit_transform(X_train)
representative_digit_idx = X_digits_dist.argmax(axis=0)
X_representative_digits = X_train[representative_digit_idx]
```

Bây giờ chúng ta hãy vẽ các **hình ảnh đại diện** này và **gán nhãn** chúng theo cách thủ công:

```
# extra code - this cell generates and saves Figure 9-13

plt.figure(figsize=(8, 2))
for index, X_representative_digit in enumerate(X_representative_digits):
    plt.subplot(k // 10, 10, index + 1)
    plt.imshow(X_representative_digit.reshape(8, 8), cmap="binary",
               interpolation="bilinear")
    plt.axis('off')

save_fig("representative_images_plot", tight_layout=False)
plt.show()
```



```
y_representative_digits = np.array([
    1, 3, 6, 0, 7, 9, 2, 4, 8, 9,
    5, 4, 7, 1, 2, 6, 1, 2, 5, 1,
    4, 1, 3, 3, 8, 8, 2, 5, 6, 9,
    1, 4, 0, 6, 8, 3, 4, 6, 7, 2,
    4, 1, 0, 7, 5, 1, 9, 9, 3, 7
])
```

Bây giờ chúng ta có một tập dữ liệu chỉ với **50 thể hiện được gán nhãn** (*labeled instances*), nhưng thay vì là các thể hiện hoàn toàn ngẫu nhiên, mỗi thể hiện trong số đó là một **hình ảnh đại diện** của cụm nó. Hãy xem liệu hiệu suất có tốt hơn không:

```
log_reg = LogisticRegression(max_iter=10_000)
log_reg.fit(X_representative_digits, y_representative_digits)
log_reg.score(X_test, y_test)
```

```
0.8488664987405542
```

Chà! Chúng ta đã **tăng vọt** từ độ chính xác 74.8% lên 84.9%, mặc dù chúng ta **vẫn chỉ huấn luyện mô hình trên 50 thể hiện**. Vì việc **gán nhãn** các thể hiện thường tốn kém và khó khăn, đặc biệt khi nó phải được thực hiện thủ công bởi các chuyên gia, nên đó là một ý tưởng

hay để khiến họ gán nhãn cho **các thể hiện đại diện** thay vì chỉ các thể hiện ngẫu nhiên.

Nhưng có lẽ chúng ta có thể tiến thêm một bước nữa: điều gì sẽ xảy ra nếu chúng ta **lan truyền** (*propagate*) các **nhãn** (*labels*) đó đến tất cả các **thể hiện** (*instances*) khác trong cùng một cụm?

```
y_train_propagated = np.empty(len(X_train), dtype=np.int64)
for i in range(k):
    y_train_propagated[kmeans.labels_ == i] = y_representative_digits[i]
```

```
log_reg = LogisticRegression(max_iter=10_000)
log_reg.fit(X_train, y_train_propagated)
```

```
LogisticRegression(max_iter=10000)
```

```
log_reg.score(X_test, y_test)
```

```
0.8942065491183879
```

Chúng ta lại có thêm một **bước tăng đáng kể về độ chính xác** (*significant accuracy boost*)! Hãy xem liệu chúng ta có thể làm tốt hơn nữa bằng cách **bỏ qua 1%** các **thể hiện** (*instances*) **xa tâm cụm** (*cluster center*) nhất của chúng: điều này sẽ giúp loại bỏ một số **điểm ngoại lai** (*outliers*):

```
percentile_closest = 99

X_cluster_dist = X_digits_dist[np.arange(len(X_train)), kmeans.labels_]
for i in range(k):
    in_cluster = (kmeans.labels_ == i)
    cluster_dist = X_cluster_dist[in_cluster]
    cutoff_distance = np.percentile(cluster_dist, percentile_closest)
    above_cutoff = (X_cluster_dist > cutoff_distance)
    X_cluster_dist[in_cluster & above_cutoff] = -1

partially_propagated = (X_cluster_dist != -1)
X_train_partially_propagated = X_train[partially_propagated]
y_train_partially_propagated = y_train_propagated[partially_propagated]
```

```
log_reg = LogisticRegression(max_iter=10_000)
log_reg.fit(X_train_partially_propagated, y_train_partially_propagated)
log_reg.score(X_test, y_test)
```

```
0.9093198992443325
```

Chà, lại thêm một **bước tăng về độ chính xác** (*accuracy boost*) nữa! Chúng ta thậm chí đã **vượt qua** một chút hiệu suất mà chúng ta có được khi **huấn luyện trên tập huấn luyện được gán nhãn đầy đủ** (*fully labeled training set*)!

Các **nhãn được lan truyền** (*propagated labels*) của chúng ta thực sự khá tốt: độ chính xác của chúng là khoảng 97.6%:

```
(y_train_partially_propagated == y_train[partially_propagated]).mean()
```

```
0.9755555555555555
```

Bây giờ bạn có thể thực hiện một vài lần **lặp** của **học tập tích cực** (*active learning*):

1. **Gán nhãn thủ công** các **thể hiện** (*instances*) mà bộ phân loại **ít chắc chắn nhất** (least sure about), nếu có thể, bằng cách chọn chúng trong các cụm khác nhau.
2. Huấn luyện một **mô hình mới** với các nhãn bổ sung này.

▼ DBSCAN

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=1000, noise=0.05, random_state=42)
dbscan = DBSCAN(eps=0.05, min_samples=5)
dbscan.fit(X)
```

```
DBSCAN(eps=0.05)
```

```
dbscan.labels_[ :10]
```

```
array([ 0,  2, -1, -1,  1,  0,  0,  0,  2,  5])
```

```
dbscan.core_sample_indices_[ :10]
```

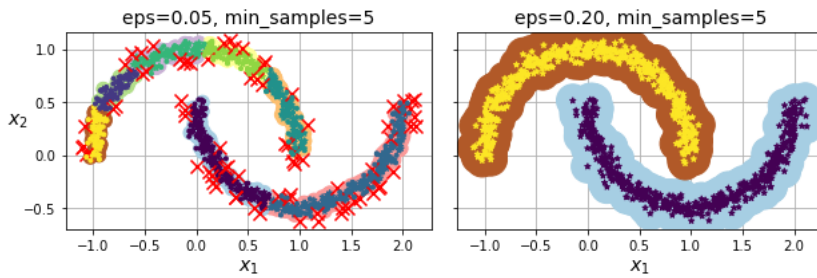
```
array([ 0,  4,  5,  6,  7,  8, 10, 11, 12, 13])
```

```
dbscan.components_
```

```
array([[ -0.02137124,  0.40618608],  
       [ -0.84192557,  0.53058695],  
       [  0.58930337, -0.32137599],  
       ...,  
       [  1.66258462, -0.3079193 ],  
       [ -0.94355873,  0.3278936 ],  
       [  0.79419406,  0.60777171]])
```

```
# extra code - this cell generates and saves Figure 9-14
```

```
def plot_dbscan(dbscan, X, size, show_xlabels=True, show_ylabels=True):  
    core_mask = np.zeros_like(dbscan.labels_, dtype=bool)  
    core_mask[dbscan.core_sample_indices_] = True  
    anomalies_mask = dbscan.labels_ == -1  
    non_core_mask = ~(core_mask | anomalies_mask)  
  
    cores = dbscan.components_  
    anomalies = X[anomalies_mask]  
    non_cores = X[non_core_mask]  
  
    plt.scatter(cores[:, 0], cores[:, 1],  
                c=dbscan.labels_[core_mask], marker='o', s=size, cmap="Paired")  
    plt.scatter(cores[:, 0], cores[:, 1], marker='*', s=20,  
                c=dbscan.labels_[core_mask])  
    plt.scatter(anomalies[:, 0], anomalies[:, 1],  
                c="r", marker="x", s=100)  
    plt.scatter(non_cores[:, 0], non_cores[:, 1],  
                c=dbscan.labels_[non_core_mask], marker=".")  
    if show_xlabels:  
        plt.xlabel("$x_1$")  
    else:  
        plt.tick_params(labelbottom=False)  
    if show_ylabels:  
        plt.ylabel("$x_2$", rotation=0)  
    else:  
        plt.tick_params(labelleft=False)  
    plt.title(f"eps={dbscan.eps:.2f}, min_samples={dbscan.min_samples}")  
    plt.grid()  
    plt.gca().set_axisbelow(True)  
  
    dbscan2 = DBSCAN(eps=0.2)  
    dbscan2.fit(X)  
  
    plt.figure(figsize=(9, 3.2))  
  
    plt.subplot(121)  
    plot_dbscan(dbscan, X, size=100)  
  
    plt.subplot(122)  
    plot_dbscan(dbscan2, X, size=600, show_ylabels=False)  
  
    save_fig("dbscan_plot")  
    plt.show()
```



```
dbscan = dbscan2 # extra code - the text says we now use eps=0.2
```

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=50)
knn.fit(dbscan.components_, dbscan.labels_[dbscan.core_sample_indices_])
```

```
KNeighborsClassifier(n_neighbors=50)
```

```
X_new = np.array([[-0.5, 0], [0, 0.5], [1, -0.1], [2, 1]])
knn.predict(X_new)
```

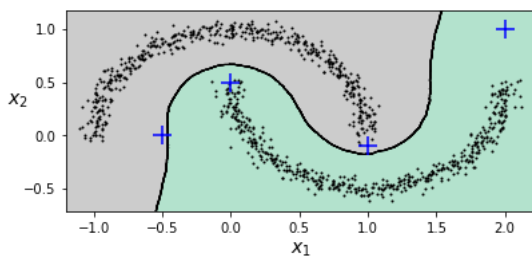
```
array([1, 0, 1, 0])
```

```
knn.predict_proba(X_new)
```

```
array([[0.18, 0.82],
       [1. , 0. ],
       [0.12, 0.88],
       [1. , 0. ]])
```

```
# extra code - this cell generates and saves Figure 9-15
```

```
plt.figure(figsize=(6, 3))
plot_decision_boundaries(knn, X, show_centroids=False)
plt.scatter(X_new[:, 0], X_new[:, 1], c="b", marker="+", s=200, zorder=10)
save_fig("cluster_classification_plot")
plt.show()
```



```
y_dist, y_pred_idx = knn.kneighbors(X_new, n_neighbors=1)
y_pred = dbscan.labels_[dbscan.core_sample_indices_][y_pred_idx]
y_pred[y_dist > 0.2] = -1
y_pred.ravel()
```

```
array([-1, 0, 1, -1])
```

✧ Các Thuật toán Phân cụm Khác

Mã trong phần này là tài liệu bổ sung, không có trong sách.

✧ Phân cụm Phổ

```
from sklearn.cluster import SpectralClustering
```

```
sc1 = SpectralClustering(n_clusters=2, gamma=100, random_state=42)
sc1.fit(X)
```

```
SpectralClustering(gamma=100, n_clusters=2, random_state=42)
```

```
sc1.affinity_matrix_.round(2)
```

```
array([[1. , 0. , 0. , ..., 0. , 0. , 0. ],
       [0. , 1. , 0.3, ..., 0. , 0. , 0. ],
       [0. , 0.3, 1. , ..., 0. , 0. , 0. ],
       ...,
       [0. , 0. , 0. , ..., 1. , 0. , 0. ],
       [0. , 0. , 0. , ..., 0. , 1. , 0. ],
       [0. , 0. , 0. , ..., 0. , 0. , 1. ]])
```

```
sc2 = SpectralClustering(n_clusters=2, gamma=1, random_state=42)
sc2.fit(X)
```

```
SpectralClustering(gamma=1, n_clusters=2, random_state=42)
```

```
def plot_spectral_clustering(sc, X, size, alpha, show_xlabels=True,
                             show_ylabels=True):
    plt.scatter(X[:, 0], X[:, 1], marker='o', s=size, c='gray', alpha=alpha)
    plt.scatter(X[:, 0], X[:, 1], marker='o', s=30, c='w')
    plt.scatter(X[:, 0], X[:, 1], marker='.', s=10, c=sc.labels_, cmap="Paired")

    if show_xlabels:
        plt.xlabel("$x_1$")
    else:
        plt.tick_params(labelbottom=False)
    if show_ylabels:
        plt.ylabel("$x_2$", rotation=0)
    else:
        plt.tick_params(labelleft=False)
    plt.title(f"RBF gamma={sc.gamma}")
```

```
plt.figure(figsize=(9, 3.2))
```

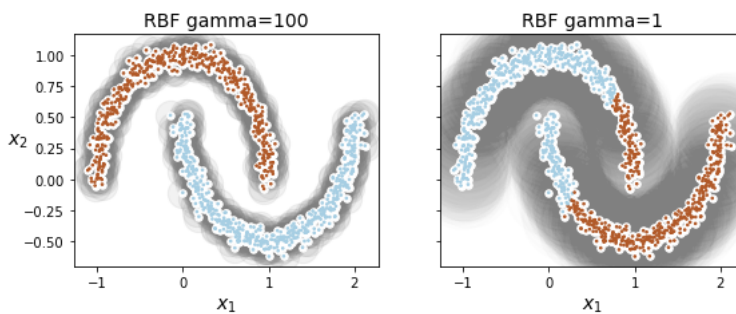
```
plt.subplot(121)
```

```
plot_spectral_clustering(sc1, X, size=500, alpha=0.1)
```

```
plt.subplot(122)
```

```
plot_spectral_clustering(sc2, X, size=4000, alpha=0.01, show_ylabels=False)
```

```
plt.show()
```



✦ Phân cụm Kết tụ

```
from sklearn.cluster import AgglomerativeClustering
```

```
X = np.array([0, 2, 5, 8.5]).reshape(-1, 1)
agg = AgglomerativeClustering(linkage="complete").fit(X)
```

```
def learned_parameters(estimator):
    return [attrib for attrib in dir(estimator)
            if attrib.endswith("_") and not attrib.startswith("_")]
```

```
learned_parameters(agg)
```

```
['children_',  
'labels_',  
'n_clusters_',  
'n_connected_components_',  
'n_features_in_',  
'n_leaves_']
```

```
agg.children_
```

```
array([[0, 1],  
       [2, 3],  
       [4, 5]])
```

▼ Hỗn hợp Gaussian

Hãy tạo cùng một tập dữ liệu như trước với ba **hình elip** (*ellipsoids*) (tập dữ liệu mà **K-Trung bình** *K-Means* đã gặp khó khăn):

```
X1, y1 = make_blobs(n_samples=1000, centers=((4, -4), (0, 0)), random_state=42)  
X1 = X1.dot(np.array([[0.374, 0.95], [0.732, 0.598]]))  
X2, y2 = make_blobs(n_samples=250, centers=1, random_state=42)  
X2 = X2 + [6, -8]  
X = np.r_[X1, X2]  
y = np.r_[y1, y2]
```

Hãy huấn luyện một **mô hình hỗn hợp Gaussian** (*Gaussian mixture model*) trên tập dữ liệu trước đó:

```
from sklearn.mixture import GaussianMixture
```

```
gm = GaussianMixture(n_components=3, n_init=10, random_state=42)  
gm.fit(X)
```

```
GaussianMixture(n_components=3, n_init=10, random_state=42)
```

Hãy xem xét các **tham số** (*parameters*) mà thuật toán **EM** đã ước tính:

```
gm.weights_
```

```
array([0.39025715, 0.40007391, 0.20966893])
```

```
gm.means_
```

```
array([[ 0.05131611,  0.07521837],  
       [-1.40763156,  1.42708225],  
       [ 3.39893794,  1.05928897]])
```

```
gm.covariances_
```

```
array([[[ 0.68799922,  0.79606357],  
        [ 0.79606357,  1.21236106]],  
       [[ 0.63479409,  0.72970799],  
        [ 0.72970799,  1.1610351 ]],  
       [[ 1.14833585, -0.03256179],  
        [-0.03256179,  0.95490931]]])
```

Thuật toán đã hội tụ chưa?

```
gm.converged_
```

```
True
```

Vâng, tốt. Nó đã mất bao nhiêu lần lặp?

```
gm.n_iter_
```

Bây giờ bạn có thể sử dụng mô hình để **dự đoán cụm** mà mỗi **thể hiện** (*instance*) thuộc về (**phân cụm cứng** - *hard clustering*) hoặc **xác suất** nó đến từ mỗi cụm. Để làm điều này, chỉ cần sử dụng phương thức `predict()` hoặc phương thức

```
gm.predict(X)
```

```
array([0, 0, 1, ..., 2, 2, 2])
```

```
gm.predict_proba(X).round(3)
```

```
array([[0.977, 0.    , 0.023],
       [0.983, 0.001, 0.016],
       [0.    , 1.    , 0.    ],
       ...,
       [0.    , 0.    , 1.    ],
       [0.    , 0.    , 1.    ],
       [0.    , 0.    , 1.    ]])
```

Đây là một **mô hình sinh** (*generative model*), vì vậy bạn có thể **lấy mẫu** (*sample*) các **thể hiện mới** (*new instances*) từ nó (và lấy **nhãn** của chúng):

```
X_new, y_new = gm.sample(6)
X_new
```

```
array([[ -0.86944074, -0.32767626],
       [ 0.29836051,  0.28297011],
       [-2.8014927 , -0.09047309],
       [ 3.98203732,  1.49951491],
       [ 3.81677148,  0.53095244],
       [ 2.84104923, -0.73858639]])
```

```
y_new
```

```
array([0, 0, 1, 2, 2, 2])
```

Lưu ý rằng chúng được lấy mẫu tuần tự (sequentially) từ mỗi cụm.

Bạn cũng có thể ước tính **log** của **hàm mật độ xác suất** (*probability density function* - *PDF*) tại bất kỳ vị trí nào bằng cách sử dụng phương thức `score_samples()`:

```
gm.score_samples(X).round(2)
```

```
array([-2.61, -3.57, -3.33, ..., -3.51, -4.4 , -3.81])
```

Hãy kiểm tra xem **Hàm mật độ xác suất** (*PDF*) có **tích phân** (*integrates*) bằng **1** trên toàn bộ không gian hay không. Chúng ta chỉ cần lấy một hình vuông lớn bao quanh các cụm, và chia nó thành một lưới các hình vuông nhỏ, sau đó chúng ta tính **xác suất xấp xỉ** (*approximate probability*) mà các thể hiện sẽ được sinh ra trong mỗi hình vuông nhỏ (bằng cách nhân **PDF** tại một góc của hình vuông nhỏ với **diện tích** của hình vuông đó), và cuối cùng **tổng hợp** (*summing*) tất cả các xác suất này. Kết quả rất **gần với 1**:

```
# extra code - bonus material
```

```
resolution = 100
grid = np.arange(-10, 10, 1 / resolution)
xx, yy = np.meshgrid(grid, grid)
X_full = np.vstack([xx.ravel(), yy.ravel()]).T
```

```
pdf = np.exp(gm.score_samples(X_full))
pdf_probas = pdf * (1 / resolution) ** 2
pdf_probas.sum()
```

```
0.9999999999215021
```

Bây giờ chúng ta hãy vẽ **ranh giới quyết định** (*decision boundaries*) thu được (các đường gạch ngang) và **đường đồng mức mật độ** (*density contours*):

```
# extra code - this cells generates and saves Figure 9-16

from matplotlib.colors import LogNorm

def plot_gaussian_mixture(clusterer, X, resolution=1000, show_ylabels=True):
    mins = X.min(axis=0) - 0.1
    maxs = X.max(axis=0) + 0.1
    xx, yy = np.meshgrid(np.linspace(mins[0], maxs[0], resolution),
                          np.linspace(mins[1], maxs[1], resolution))
    Z = -clusterer.score_samples(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z,
                 norm=LogNorm(vmin=1.0, vmax=30.0),
                 levels=np.logspace(0, 2, 12))
    plt.contour(xx, yy, Z,
               norm=LogNorm(vmin=1.0, vmax=30.0),
               levels=np.logspace(0, 2, 12),
               linewidths=1, colors='k')

    Z = clusterer.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    plt.contour(xx, yy, Z,
               linewidths=2, colors='r', linestyle='dashed')

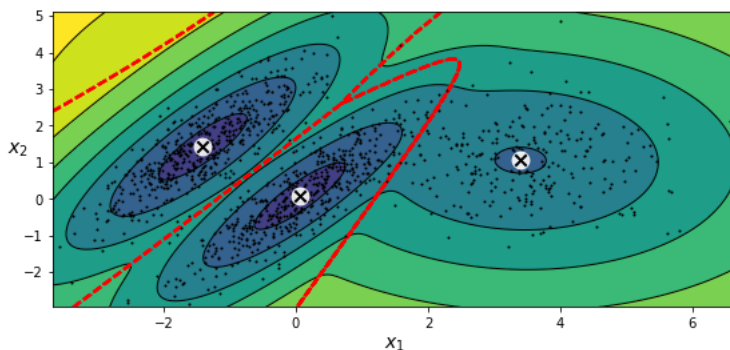
    plt.plot(X[:, 0], X[:, 1], 'k.', markersize=2)
    plot_centroids(clusterer.means_, clusterer.weights_)

    plt.xlabel("$x_1$")
    if show_ylabels:
        plt.ylabel("$x_2$", rotation=0)
    else:
        plt.tick_params(labelleft=False)

plt.figure(figsize=(8, 4))

plot_gaussian_mixture(gm, X)

save_fig("gaussian_mixtures_plot")
plt.show()
```



Bạn có thể **áp đặt các ràng buộc** lên các **ma trận hiệp phương sai** (*covariance matrices*) mà thuật toán tìm kiếm bằng cách đặt siêu tham số `covariance_type`:

- **"spherical"**: tất cả các cụm phải có **dạng hình cầu**, nhưng chúng có thể có các đường kính khác nhau (tức là **phương sai** khác nhau).
- **"diag"**: các cụm có thể mang bất kỳ **hình elip** nào với bất kỳ kích thước nào, nhưng các trục của hình elip phải **song song** với các **trục tọa độ** (tức là các **ma trận hiệp phương sai phải là ma trận đường chéo**).
- **"tied"**: tất cả các cụm phải có **cùng hình dạng**, vốn có thể là bất kỳ hình elip nào (tức là tất cả chúng **chia sẻ chung một ma trận hiệp phương sai**).
- **"full"** (mặc định): **không có ràng buộc**, tất cả các cụm có thể mang bất kỳ hình elip nào với bất kỳ kích thước nào.

```
# extra code - this cell generates and saves Figure 9-17

gm_full = GaussianMixture(n_components=3, n_init=10,
                          covariance_type="full", random_state=42)
gm_tied = GaussianMixture(n_components=3, n_init=10,
                          covariance_type="tied", random_state=42)
```



```

gm_spherical = GaussianMixture(n_components=3, n_init=10,
                                covariance_type="spherical", random_state=42)
gm_diag = GaussianMixture(n_components=3, n_init=10,
                           covariance_type="diag", random_state=42)

gm_full.fit(X)
gm_tied.fit(X)
gm_spherical.fit(X)
gm_diag.fit(X)

def compare_gaussian_mixtures(gm1, gm2, X):
    plt.figure(figsize=(9, 4))

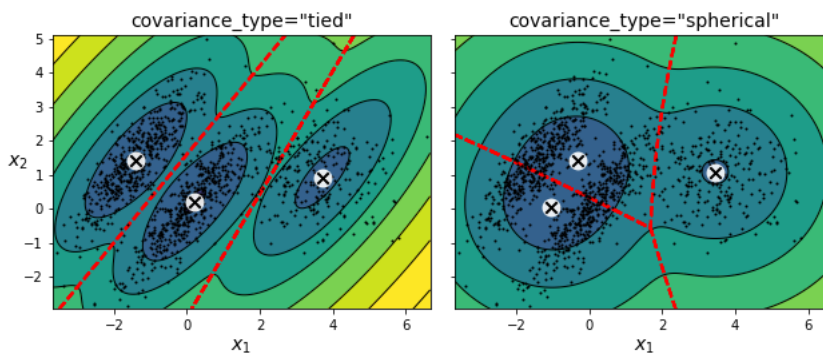
    plt.subplot(121)
    plot_gaussian_mixture(gm1, X)
    plt.title(f'covariance_type="{gm1.covariance_type}"')

    plt.subplot(122)
    plot_gaussian_mixture(gm2, X, show_ylabels=False)
    plt.title(f'covariance_type="{gm2.covariance_type}"')

compare_gaussian_mixtures(gm_tied, gm_spherical, X)

save_fig("covariance_type_plot")
plt.show()

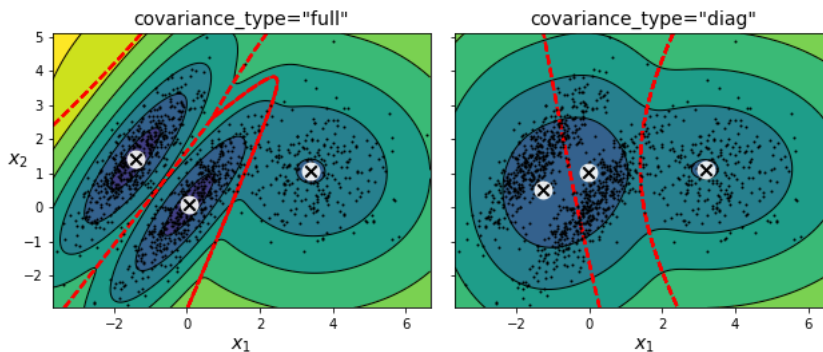
```



```

# extra code - comparing covariance_type="full" and covariance_type="diag"
compare_gaussian_mixtures(gm_full, gm_diag, X)
plt.tight_layout()
plt.show()

```



✧ Phát hiện Bất thường bằng cách sử dụng Hỗn hợp Gaussian

Hỗn hợp Gaussian (*Gaussian Mixtures*) có thể được sử dụng cho **phát hiện bất thường** (*anomaly detection*): các **thể hiện** (*instances*) nằm ở các vùng có **mật độ thấp** (*low-density regions*) có thể được coi là **bất thường** (*anomalies*). Bạn phải xác định **ngưỡng mật độ** (*density threshold*) mà bạn muốn sử dụng. Ví dụ: trong một công ty sản xuất cố gắng phát hiện các sản phẩm lỗi, tỷ lệ sản phẩm lỗi thường được biết rõ. Giả sử tỷ lệ đó bằng **2%**, thì bạn có thể đặt ngưỡng mật độ là giá trị dẫn đến việc có **2%** số thể hiện nằm trong các khu vực **dưới ngưỡng mật độ** đó:

```

densities = gm.score_samples(X)
density_threshold = np.percentile(densities, 2)
anomalies = X[densities < density_threshold]

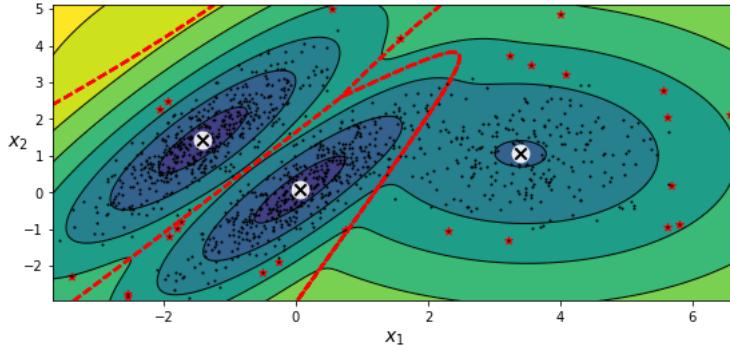
```

```
# extra code - this cell generates and saves Figure 9-18

plt.figure(figsize=(8, 4))

plot_gaussian_mixture(gm, X)
plt.scatter(anomalies[:, 0], anomalies[:, 1], color='r', marker='*')
plt.ylim(top=5.1)

save_fig("mixture_anomaly_detection_plot")
plt.show()
```



✓ Chọn Số lượng Cụm

Chúng ta **không thể sử dụng quán tính** (*inertia*) hay **điểm số hình bóng** (*silhouette score*) vì cả hai đều giả định rằng các cụm có **dạng hình cầu** (*spherical*). Thay vào đó, chúng ta có thể cố gắng tìm mô hình **giảm thiểu** (*minimizes*) một tiêu chí thông tin lý thuyết như **Tiêu chí Thông tin Bayesian** (*Bayesian Information Criterion - BIC*) hoặc **Tiêu chí Thông tin Akaike** (*Akaike Information Criterion - AIC*):

$$BIC = \log(m)p - 2\log(\hat{L})$$

$$AIC = 2p - 2\log(\hat{L})$$

- m là số lượng **thể hiện** (*instances*).
- p là số lượng **tham số** (*parameters*) được mô hình học.
- $\hat{L}$ là giá trị **cực đại** (*maximized value*) của **hàm khả năng hợp lý** (*likelihood function*) của mô hình. Đây là **xác suất có điều kiện** (*conditional probability*) của dữ liệu quan sát được $\mathbf{X}$, với mô hình và các tham số đã được tối ưu hóa của nó.

Cả **BIC** và **AIC** đều **phạt** (*penalize*) các mô hình có nhiều tham số để học (ví dụ: nhiều cụm hơn) và **thưởng** (*reward*) cho các mô hình khớp dữ liệu tốt (tức là các mô hình cung cấp **khả năng hợp lý cao** *high likelihood* cho dữ liệu quan sát được).

```
# extra code - this cell generates and saves Figure 9-19

from scipy.stats import norm

x_val = 2.5
std_val = 1.3
x_range = [-6, 4]
x_proba_range = [-2, 2]
stds_range = [1, 2]

xs = np.linspace(x_range[0], x_range[1], 501)
stds = np.linspace(stds_range[0], stds_range[1], 501)
Xs, Stds = np.meshgrid(xs, stds)
Z = 2 * norm.pdf(Xs - 1.0, 0, Stds) + norm.pdf(Xs + 4.0, 0, Stds)
Z = Z / Z.sum(axis=1)[:, np.newaxis] / (xs[1] - xs[0])

x_example_idx = (xs >= x_val).argmax() # index of the first value >= x_val
max_idx = Z[:, x_example_idx].argmax()
max_val = Z[:, x_example_idx].max()
s_example_idx = (stds >= std_val).argmax()
x_range_min_idx = (xs >= x_proba_range[0]).argmax()
x_range_max_idx = (xs >= x_proba_range[1]).argmax()
log_max_idx = np.log(Z[:, x_example_idx]).argmax()
log_max_val = np.log(Z[:, x_example_idx]).max()

plt.figure(figsize=(8, 4.5))
```

```

plt.subplot(2, 2, 1)
plt.contourf(Xs, Stds, Z, cmap="GnBu")
plt.plot([-6, 4], [std_val, std_val], "k-", linewidth=2)
plt.plot([x_val, x_val], [1, 2], "b-", linewidth=2)
plt.ylabel(r"$\theta$", rotation=0, labelpad=10)
plt.title(r"Model $f(x; \theta)$")

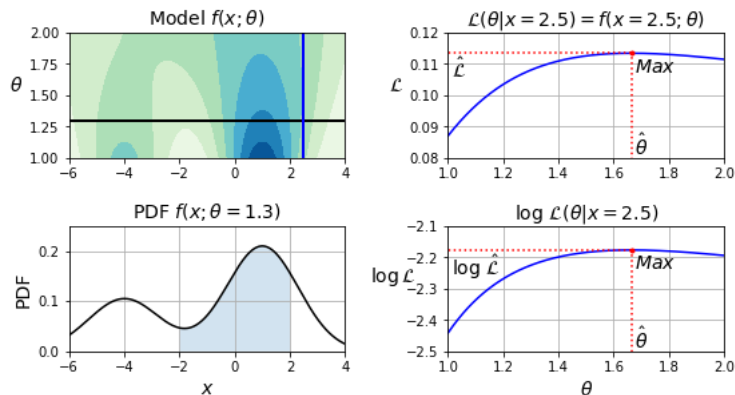
plt.subplot(2, 2, 2)
plt.plot(stds, Z[:, x_example_idx], "b-")
plt.plot(stds[max_idx], max_val, "r.")
plt.plot([stds[max_idx], stds[max_idx]], [0, max_val], "r:")
plt.plot([0, stds[max_idx]], [max_val, max_val], "r:")
plt.text(stds[max_idx]+ 0.01, 0.081, r"$\hat{\theta}$")
plt.text(stds[max_idx]+ 0.01, max_val - 0.006, r"$Max$")
plt.text(1.01, max_val - 0.008, r"$\hat{\mathcal{L}}$")
plt.ylabel(r"$\mathcal{L}$", rotation=0, labelpad=10)
plt.title(fr"$\mathcal{L}(\theta|x=\{x_val\}) = f(x=\{x_val\}; \theta)$")
plt.grid()
plt.axis([1, 2, 0.08, 0.12])

plt.subplot(2, 2, 3)
plt.plot(xs, Z[s_example_idx], "k-")
plt.fill_between(xs[x_range_min_idx:x_range_max_idx+1],
                 Z[s_example_idx, x_range_min_idx:x_range_max_idx+1], alpha=0.2)
plt.xlabel(r"$x$")
plt.ylabel("PDF")
plt.title(fr"PDF $f(x; \theta=\{std_val\})$")
plt.grid()
plt.axis([-6, 4, 0, 0.25])

plt.subplot(2, 2, 4)
plt.plot(stds, np.log(Z[:, x_example_idx]), "b-")
plt.plot(stds[log_max_idx], log_max_val, "r.")
plt.plot([stds[log_max_idx], stds[log_max_idx]], [-5, log_max_val], "r:")
plt.plot([0, stds[log_max_idx]], [log_max_val, log_max_val], "r:")
plt.text(stds[log_max_idx]+ 0.01, log_max_val - 0.06, r"$Max$")
plt.text(stds[log_max_idx]+ 0.01, -2.49, r"$\hat{\theta}$")
plt.text(1.01, log_max_val - 0.08, r"$\log \hat{\mathcal{L}}$")
plt.xlabel(r"$\theta$")
plt.ylabel(r"$\log \mathcal{L}$", rotation=0, labelpad=10)
plt.title(fr"$\log \mathcal{L}(\theta|x=\{x_val\})$")
plt.grid()
plt.axis([1, 2, -2.5, -2.1])

save_fig("likelihood_function_plot")
plt.show()

```



```
gm.bic(X)
```

```
8189.747000497186
```

```
gm.aic(X)
```

```
8102.521720382148
```

Chúng ta có thể tính toán **BIC** (*Bayesian Information Criterion*) theo cách thủ công như sau:

```
# extra code - bonus material
n_clusters = 3
n_dims = 2
n_params_for_weights = n_clusters - 1
n_params_for_means = n_clusters * n_dims
n_params_for_covariance = n_clusters * n_dims * (n_dims + 1) // 2
n_params = n_params_for_weights + n_params_for_means + n_params_for_covariance
max_log_likelihood = gm.score(X) * len(X) # log(L^)
bic = np.log(len(X)) * n_params - 2 * max_log_likelihood
aic = 2 * n_params - 2 * max_log_likelihood
print(f"bic = {bic}")
print(f"aic = {aic}")
print(f"n_params = {n_params}")
```

```
bic = 8189.747000497186
aic = 8102.521720382148
n_params = 17
```

Có **một trọng số** (*weight*) cho mỗi cụm, nhưng tổng của chúng phải bằng 1, vì vậy chúng ta có **một bậc tự do** (*degree of freedom*) ít hơn, do đó là **-1**. Tương tự, **bậc tự do** cho một **ma trận hiệp phương sai** (*covariance matrix*) $n \times n$ không phải là n^2 , mà là

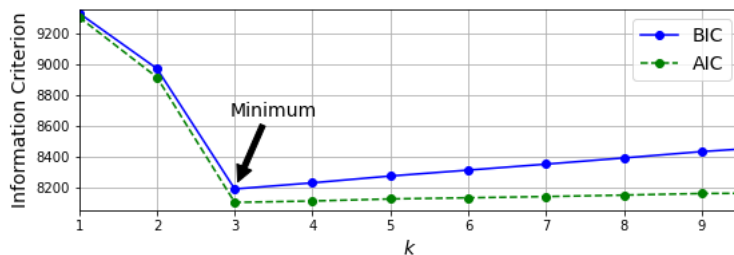
$$1 + 2 + \dots + n = \frac{n(n+1)}{2}.$$

Hãy huấn luyện các **mô hình Hỗn hợp Gaussian** (*Gaussian Mixture models*) với các giá trị k khác nhau và **đo lường** (*measure*) **BIC** (*Bayesian Information Criterion*) của chúng:

```
# extra code - this cell generates and saves Figure 9-20

gms_per_k = [GaussianMixture(n_components=k, n_init=10, random_state=42).fit(X)
              for k in range(1, 11)]
bics = [model.bic(X) for model in gms_per_k]
aics = [model.aic(X) for model in gms_per_k]

plt.figure(figsize=(8, 3))
plt.plot(range(1, 11), bics, "bo-", label="BIC")
plt.plot(range(1, 11), aics, "go--", label="AIC")
plt.xlabel("$k$")
plt.ylabel("Information Criterion")
plt.axis([1, 9.5, min(aics) - 50, max(aics) + 50])
plt.annotate("", xy=(3, bics[2]), xytext=(3.4, 8650),
             arrowprops=dict(facecolor='black', shrink=0.1))
plt.text(3.5, 8660, "Minimum", horizontalalignment="center")
plt.legend()
plt.grid()
save_fig("aic_bic_vs_k_plot")
plt.show()
```



✧ Mô hình Hỗn hợp Gaussian Bayes

Thay vì tự mình tìm kiếm **số lượng cụm tối ưu** (*optimal number of clusters*), ta có thể sử dụng lớp `BayesianGaussianMixture` vốn có khả năng đặt các **trọng số** (*weights*) bằng (hoặc gần) không cho các cụm **không cần thiết**. Chỉ cần đặt **số lượng thành phần** (*number of components*) thành một giá trị mà bạn tin rằng **lớn hơn** số lượng cụm tối ưu, và thuật toán sẽ tự động loại bỏ các cụm không cần thiết.

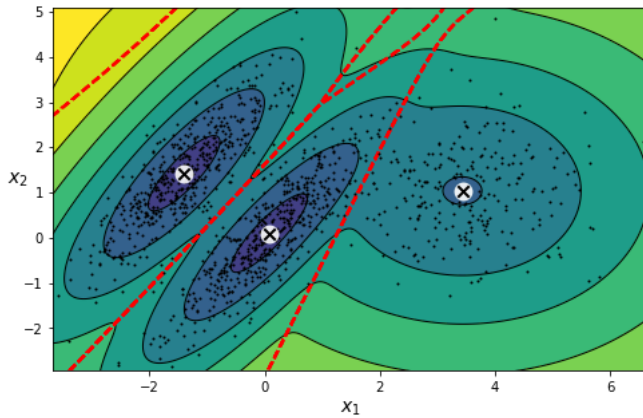
```
from sklearn.mixture import BayesianGaussianMixture

bgm = BayesianGaussianMixture(n_components=10, n_init=10, random_state=42)
bgm.fit(X)
bgm.weights_.round(2)
```

```
array([0.4 , 0.21, 0.4 , 0. , 0. , 0. , 0. , 0. , 0. , 0. ])
```

Thuật toán đã **tự động phát hiện** rằng chỉ **3 thành phần** (*components*) là cần thiết!

```
# extra code - this figure is almost identical to Figure 9-16
plt.figure(figsize=(8, 5))
plot_gaussian_mixture(bgm, X)
plt.show()
```



```
# extra code - this cell generates and saves Figure 9-21

X_moons, y_moons = make_moons(n_samples=1000, noise=0.05, random_state=42)

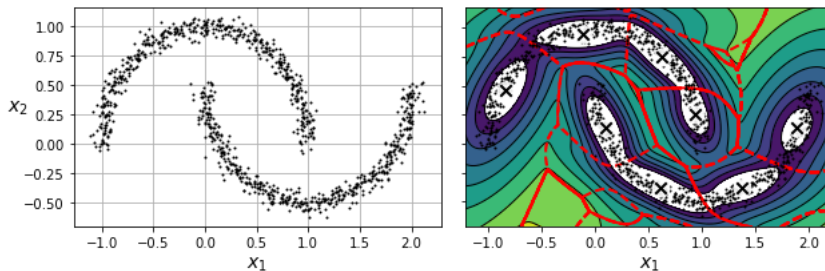
bgm = BayesianGaussianMixture(n_components=10, n_init=10, random_state=42)
bgm.fit(X_moons)

plt.figure(figsize=(9, 3.2))

plt.subplot(121)
plot_data(X_moons)
plt.xlabel("$x_1$")
plt.ylabel("$x_2$", rotation=0)
plt.grid()

plt.subplot(122)
plot_gaussian_mixture(bgm, X_moons, show_ylabels=False)

save_fig("moons_vs_bgm_plot")
plt.show()
```



Ồi, không ổn... thay vì phát hiện **2 cụm hình mặt trăng**, thuật toán đã phát hiện **8 cụm hình elip**. Tuy nhiên, **biểu đồ mật độ** (*density plot*) trông không quá tệ, vì vậy nó có thể sử dụng được cho **phát hiện bất thường** (*anomaly detection*).

✓ Hướng dẫn bài tập

✓ 1. tới 9.

1. Trong Học máy (*Machine Learning*), **phân cụm** (*clustering*) là nhiệm vụ **không giám sát** (*unsupervised task*) nhằm nhóm các **thể hiện** (*instances*) tương tự nhau lại. Khái niệm về **sự tương đồng** (*similarity*) phụ thuộc vào nhiệm vụ đang thực hiện: ví dụ, trong một số trường hợp, hai thể hiện gần nhau sẽ được coi là tương tự, trong khi ở những trường hợp khác, các thể hiện tương tự có thể cách xa nhau miễn là chúng thuộc về cùng một nhóm **được đóng gói dày đặc** (*densely packed group*). Các thuật toán phân cụm phổ biến bao gồm **K-Trung bình** (*K-Means*), **DBSCAN**, **phân cụm kết tụ** (*agglomerative clustering*), **BIRCH**, **Dịch chuyển Trung bình** (*Mean-Shift*), **lan truyền quan hệ** (*affinity propagation*), và **phân cụm phổ** (*spectral clustering*).
2. Các ứng dụng chính của thuật toán phân cụm bao gồm **phân tích dữ liệu**, **phân đoạn khách hàng** (*customer segmentation*), **hệ thống đề xuất** (*recommender systems*), **công cụ tìm kiếm**, **phân đoạn ảnh** (*image segmentation*), **học bán giám sát** (*semi-supervised learning*), **giảm chiều** (*dimensionality reduction*), **phát hiện bất thường** (*anomaly detection*), và **phát hiện tính mới** (*novelty detection*).
3. **Quy tắc cùi chỏ** (*elbow rule*) là một kỹ thuật đơn giản để chọn số lượng cụm khi sử dụng **K-Trung bình**: chỉ cần vẽ **quán tính** (*inertia*) (khoảng cách bình phương trung bình từ mỗi thể hiện đến **tâm cụm** *centroid* gần nhất của nó) theo hàm của số lượng cụm, và tìm điểm trên đường cong nơi quán tính **ngừng giảm nhanh** ("cùi chỏ"). Điểm này thường gắn với số lượng cụm tối ưu. Một phương pháp khác là vẽ **điểm số hình bóng** (*silhouette score*) theo hàm của số lượng cụm. Thường sẽ có một **đỉnh**, và số lượng cụm tối ưu thường nằm gần đó. **Điểm số hình bóng** là **hệ số hình bóng** (*silhouette coefficient*) trung bình trên tất cả các thể hiện. Hệ số này thay đổi từ **+1** cho các thể hiện nằm rõ ràng bên trong cụm của chúng và xa các cụm khác, đến **-1** cho các thể hiện rất gần với một cụm khác. Bạn cũng có thể vẽ **sơ đồ hình bóng** (*silhouette diagrams*) và thực hiện một phân tích kỹ lưỡng hơn.
4. **Gán nhãn** một tập dữ liệu rất tốn kém và mất thời gian. Do đó, việc có nhiều **thể hiện không được gán nhãn** (*unlabeled instances*) nhưng ít **thể hiện được gán nhãn** (*labeled instances*) là điều phổ biến. **Lan truyền nhãn** (*Label propagation*) là một kỹ thuật bao gồm việc **sao chép** một số (hoặc tất cả) các nhãn từ các thể hiện đã được gán nhãn sang các thể hiện không được gán nhãn tương tự. Điều này có thể mở rộng đáng kể số lượng thể hiện được gán nhãn, và qua đó cho phép một thuật toán có giám sát đạt được hiệu suất tốt hơn (đây là một hình thức của **học bán giám sát** *semi-supervised learning*). Một phương pháp là sử dụng một thuật toán phân cụm như **K-Trung bình** trên tất cả các thể hiện, sau đó đối với mỗi cụm, tìm nhãn phổ biến nhất hoặc nhãn của thể hiện đại diện nhất (tức là thể hiện gần **tâm cụm** *centroid* nhất) và **lan truyền** nó đến các thể hiện không được gán nhãn trong cùng một cụm.
5. **K-Trung bình** (*K-Means*) và **BIRCH** mở rộng tốt (*scale well*) với các tập dữ liệu lớn. **DBSCAN** và **Dịch chuyển Trung bình** (*Mean-Shift*) tìm kiếm các vùng có **mật độ cao** (*high density*).
6. **Học tập tích cực** (*Active learning*) hữu ích bất cứ khi nào bạn có nhiều thể hiện không được gán nhãn nhưng việc gán nhãn lại tốn kém. Trong trường hợp này (rất phổ biến), thay vì chọn ngẫu nhiên các thể hiện để gán nhãn, việc thực hiện **học tập tích cực** thường được ưu tiên hơn, nơi các chuyên gia con người tương tác với thuật toán học, cung cấp nhãn cho các thể hiện cụ thể khi thuật toán yêu cầu. Một phương pháp tiếp cận phổ biến là **lấy mẫu không chắc chắn** (*uncertainty sampling*) (xem phần *Active Learning* trong chương 9).
7. Nhiều người sử dụng các thuật ngữ **phát hiện bất thường** (*anomaly detection*) và **phát hiện tính mới** (*novelty detection*) thay thế cho nhau, nhưng chúng không hoàn toàn giống nhau. Trong **phát hiện bất thường**, thuật toán được huấn luyện trên một tập dữ liệu **có thể chứa các điểm ngoại lai** (*outliers*), và mục tiêu thường là xác định các điểm ngoại lai này (trong tập huấn luyện), cũng như các điểm ngoại lai trong số các thể hiện mới. Trong **phát hiện tính mới**, thuật toán được huấn luyện trên một tập dữ liệu được cho là **"sạch"** (*"clean"*), và mục tiêu là phát hiện tính mới (*novelties*) **nghe nghiêm ngặt chỉ trong số các thể hiện mới**. Một số thuật toán hoạt động tốt nhất cho phát hiện bất thường (ví dụ: **Isolation Forest**), trong khi những thuật toán khác phù hợp hơn cho phát hiện tính mới (ví dụ: **one-class SVM**).
8. **Mô hình hỗn hợp Gaussian** (*Gaussian mixture model - GMM*) là một **mô hình xác suất** (*probabilistic model*) giả định rằng các thể hiện được sinh ra từ một **hỗn hợp** của một số **phân phối Gaussian** (*Gaussian distributions*) có tham số chưa được biết. Nói cách khác, giả định là dữ liệu được nhóm thành một số hữu hạn các cụm, mỗi cụm có **hình dạng elip** (*ellipsoidal shape*) (nhưng các cụm có thể có hình dạng, kích thước, định hướng và mật độ elip khác nhau), và chúng ta không biết mỗi thể hiện thuộc về cụm nào. Mô hình này hữu ích cho **ước tính mật độ** (*density estimation*), **phân cụm** (*clustering*), và **phát hiện bất thường** (*anomaly detection*).
9. Một cách để tìm ra số lượng cụm chính xác khi sử dụng **mô hình hỗn hợp Gaussian** là vẽ **tiêu chí thông tin Bayesian** (*Bayesian information criterion - BIC*) hoặc **tiêu chí thông tin Akaike** (*Akaike information criterion - AIC*) theo hàm của số lượng cụm, sau đó chọn số lượng cụm **giảm thiểu** (*minimizes*) **BIC** hoặc **AIC**. Một kỹ thuật khác là sử dụng **mô hình hỗn hợp Gaussian Bayes** (*Bayesian Gaussian mixture model*), vốn sẽ tự động chọn số lượng cụm.

✓ 10. Phân cụm Tập dữ liệu Khuôn mặt Olivetti

Bài tập: Tập dữ liệu khuôn mặt **Olivetti** cổ điển chứa 400 hình ảnh khuôn mặt **mức xám** (*grayscale*) **64 × 64 pixel**. Mỗi hình ảnh được **làm phẳng** (*flattened*) thành một **vector 1D** có kích thước **4.096**. **40 người khác nhau** đã được chụp ảnh (**10 lần mỗi người**), và nhiệm vụ thông thường là huấn luyện một mô hình có thể dự đoán người nào được đại diện trong mỗi bức ảnh. Tải tập dữ liệu bằng cách sử dụng hàm `sklearn.datasets.fetch_olivetti_faces()`.

```
from sklearn.datasets import fetch_olivetti_faces

olivetti = fetch_olivetti_faces()
```

```
print(olivetti.DESCR)
```

```
.. _olivetti_faces_dataset:
```

```
The Olivetti faces dataset
-----
```

```
`This dataset contains a set of face images`_ taken between April 1992 and
April 1994 at AT&T Laboratories Cambridge. The
:func:`sklearn.datasets.fetch_olivetti_faces` function is the data
fetching / caching function that downloads the data
archive from AT&T.
```

```
.. _This dataset contains a set of face images: http://www.cl.cam.ac.uk/research/dtg/attarchive/facedatabase.html
```

```
As described on the original website:
```

There are ten different images of each of 40 distinct subjects. For some subjects, the images were taken at different times, varying the lighting, facial expressions (open / closed eyes, smiling / not smiling) and facial details (glasses / no glasses). All the images were taken against a dark homogeneous background with the subjects in an upright, frontal position (with tolerance for some side movement).

```
**Data Set Characteristics:**
```

```
=====
Classes                40
Samples total          400
Dimensionality         4096
Features               real, between 0 and 1
=====
```

The image is quantized to 256 grey levels and stored as unsigned 8-bit integers; the loader will convert these to floating point values on the interval [0, 1], which are easier to work with for many algorithms.

The "target" for this database is an integer from 0 to 39 indicating the identity of the person pictured; however, with only 10 examples per class, this relatively small dataset is more interesting from an unsupervised or semi-supervised perspective.

The original dataset consisted of 92 x 112, while the version available here consists of 64x64 images.

When using these images, please give credit to AT&T Laboratories Cambridge.

```
olivetti.target
```

```
array([ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  1,  1,  1,  1,  1,  1,  1,
        1,  1,  1,  2,  2,  2,  2,  2,  2,  2,  2,  2,  2,  3,  3,  3,  3,
        3,  3,  3,  3,  3,  3,  4,  4,  4,  4,  4,  4,  4,  4,  4,  4,  5,
        5,  5,  5,  5,  5,  5,  5,  5,  5,  5,  6,  6,  6,  6,  6,  6,  6,  6,
        6,  6,  7,  7,  7,  7,  7,  7,  7,  7,  7,  8,  8,  8,  8,  8,  8,
        8,  8,  8,  8,  8,  9,  9,  9,  9,  9,  9,  9,  9,  9,  9, 10, 10,
       10, 10, 10, 10, 10, 10, 10, 10, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11,
       11, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 13, 13, 13, 13, 13, 13,
       13, 13, 13, 14, 14, 14, 14, 14, 14, 14, 14, 14, 14, 15, 15, 15,
       15, 15, 15, 15, 15, 15, 16, 16, 16, 16, 16, 16, 16, 16, 16, 16, 16,
       17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 18, 18, 18, 18, 18, 18, 18, 18,
       18, 18, 18, 19, 19, 19, 19, 19, 19, 19, 19, 19, 19, 20, 20, 20, 20,
       20, 20, 20, 20, 20, 20, 21, 21, 21, 21, 21, 21, 21, 21, 21, 21, 22,
       22, 22, 22, 22, 22, 22, 22, 22, 23, 23, 23, 23, 23, 23, 23, 23,
       23, 23, 24, 24, 24, 24, 24, 24, 24, 24, 24, 24, 25, 25, 25, 25, 25,
       25, 25, 25, 25, 25, 26, 26, 26, 26, 26, 26, 26, 26, 26, 26, 27, 27,
       27, 27, 27, 27, 27, 27, 27, 27, 28, 28, 28, 28, 28, 28, 28, 28, 28,
       28, 28, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 30, 30, 30, 30, 30, 30,
       30, 30, 30, 30, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32,
       32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 33, 33, 33, 33, 33, 33,
       34, 34, 34, 34, 34, 34, 34, 34, 34, 34, 35, 35, 35, 35, 35, 35, 35,
       35, 35, 35, 36, 36, 36, 36, 36, 36, 36, 36, 36, 36, 37, 37, 37, 37,
       37, 37, 37, 37, 37, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 39,
       39, 39, 39, 39, 39, 39, 39, 39, 39])
```

Bài tập: Sau đó, **chia** nó thành **tập huấn luyện** (training set), **tập xác thực** (validation set), và **tập kiểm thử** (test set) (lưu ý rằng tập dữ liệu đã được **chia tỷ lệ** (scaled) từ 0 đến 1). Vì tập dữ liệu khá nhỏ, bạn có thể muốn sử dụng **lấy mẫu phân tầng** (stratified sampling) để

đảm bảo rằng có **số lượng ảnh trên mỗi người là như nhau** trong mỗi tập.

```
from sklearn.model_selection import StratifiedShuffleSplit

strat_split = StratifiedShuffleSplit(n_splits=1, test_size=40, random_state=42)
train_valid_idx, test_idx = next(strat_split.split(olivetti.data,
                                                  olivetti.target))

X_train_valid = olivetti.data[train_valid_idx]
y_train_valid = olivetti.target[train_valid_idx]
X_test = olivetti.data[test_idx]
y_test = olivetti.target[test_idx]

strat_split = StratifiedShuffleSplit(n_splits=1, test_size=80, random_state=43)
train_idx, valid_idx = next(strat_split.split(X_train_valid, y_train_valid))
X_train = X_train_valid[train_idx]
y_train = y_train_valid[train_idx]
X_valid = X_train_valid[valid_idx]
y_valid = y_train_valid[valid_idx]
```

```
print(X_train.shape, y_train.shape)
print(X_valid.shape, y_valid.shape)
print(X_test.shape, y_test.shape)
```

```
(280, 4096) (280,)
(80, 4096) (80,)
(40, 4096) (40,)
```

Để **tăng tốc** (*speed up*) mọi thứ, chúng ta sẽ **giảm chiều dữ liệu** (*reduce the data's dimensionality*) bằng cách sử dụng **PCA**:

```
from sklearn.decomposition import PCA

pca = PCA(0.99)
X_train_pca = pca.fit_transform(X_train)
X_valid_pca = pca.transform(X_valid)
X_test_pca = pca.transform(X_test)

pca.n_components_
```

```
199
```

Bài tập: Tiếp theo, **phân cụm** (*cluster*) các hình ảnh bằng cách sử dụng **K-Trung bình** (*K-Means*), và đảm bảo rằng bạn có được **số lượng cụm tốt** (*good number of clusters*) (bằng cách sử dụng một trong các kỹ thuật đã được thảo luận trong chương này).

```
from sklearn.cluster import KMeans

k_range = range(5, 150, 5)
kmeans_per_k = []
for k in k_range:
    print(f"k={k}")
    kmeans = KMeans(n_clusters=k, n_init=10, random_state=42)
    kmeans.fit(X_train_pca)
    kmeans_per_k.append(kmeans)
```

```
k=5
k=10
k=15
k=20
k=25
k=30
k=35
k=40
k=45
k=50
k=55
k=60
k=65
k=70
k=75
k=80
k=85
k=90
k=95
k=100
k=105
k=110
```

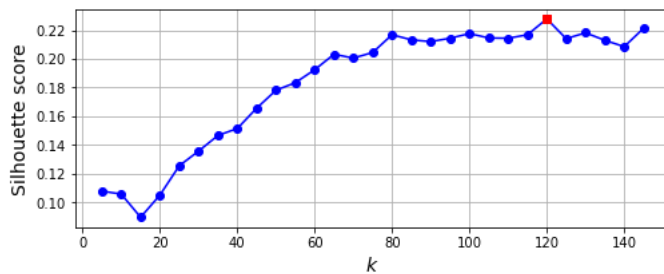


```
k=115
k=120
k=125
k=130
k=135
k=140
k=145
```

```
from sklearn.metrics import silhouette_score

silhouette_scores = [silhouette_score(X_train_pca, model.labels_)
                     for model in kmeans_per_k]
best_index = np.argmax(silhouette_scores)
best_k = k_range[best_index]
best_score = silhouette_scores[best_index]

plt.figure(figsize=(8, 3))
plt.plot(k_range, silhouette_scores, "bo-")
plt.xlabel("$k$")
plt.ylabel("Silhouette score")
plt.plot(best_k, best_score, "rs")
plt.grid()
plt.show()
```



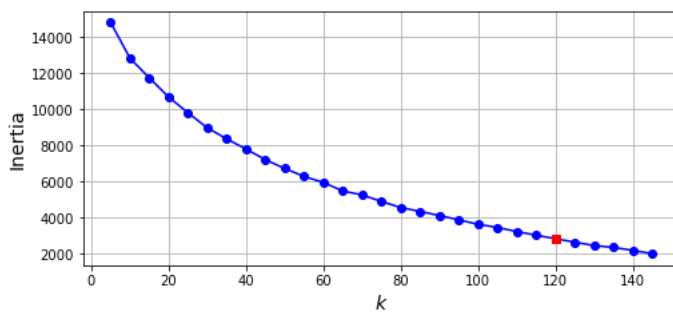
best_k

120

Có vẻ như **số lượng cụm tốt nhất** là khá cao, ở mức **120**. Bạn có thể đã mong đợi nó là **40**, vì có **40 người khác nhau** trong các bức ảnh. Tuy nhiên, cùng một người có thể trông khá khác biệt trong các bức ảnh khác nhau (ví dụ: có hoặc không có kính, hoặc chỉ đơn giản là bị dịch chuyển sang trái hoặc phải).

```
inertias = [model.inertia_ for model in kmeans_per_k]
best_inertia = inertias[best_index]

plt.figure(figsize=(8, 3.5))
plt.plot(k_range, inertias, "bo-")
plt.xlabel("$k$")
plt.ylabel("Inertia")
plt.plot(best_k, best_inertia, "rs")
plt.grid()
plt.show()
```



Số lượng cụm tối ưu không rõ ràng trên **sơ đồ quán tính** (*inertia diagram*) này, vì **không có cùi chỏ** (*elbow*) rõ ràng nào, vì vậy chúng ta hãy giữ nguyên $k = 120$.

```
best_model = kmeans_per_k[best_index]
```

Bài tập: **Trực quan hóa** (Visualize) các **cụm** (clusters): bạn có thấy các **khôn mặt tương tự** (similar faces) trong mỗi cụm không?

```
def plot_faces(faces, labels, n_cols=5):
    faces = faces.reshape(-1, 64, 64)
    n_rows = (len(faces) - 1) // n_cols + 1
    plt.figure(figsize=(n_cols, n_rows * 1.1))
    for index, (face, label) in enumerate(zip(faces, labels)):
        plt.subplot(n_rows, n_cols, index + 1)
        plt.imshow(face, cmap="gray")
        plt.axis("off")
        plt.title(label)
    plt.show()

for cluster_id in np.unique(best_model.labels_):
    print("Cluster", cluster_id)
    in_cluster = best_model.labels_==cluster_id
    faces = X_train[in_cluster]
    labels = y_train[in_cluster]
    plot_faces(faces, labels)
```


Cluster 0

28 28 28



Cluster 1

37



Cluster 2

24 22 22 22



Cluster 3

10 10 10 10



Cluster 4

12



Cluster 5

38 38 38 38 38



28



Cluster 6

23 23 23 23



Cluster 7

18



Cluster 8

29 39



Cluster 9

17



Cluster 10

37 37



Cluster 11

39 39 39



Cluster 12

33 33



Cluster 13

14 14 12 14



Cluster 14

2 25





Cluster 15

24 24 24 24



Cluster 16

7 7 7



Cluster 17

29 29 29 29 29



29



Cluster 18

39 3



Cluster 19

5 5 5 5 5



Cluster 20

27 27 27



Cluster 21

1 1 1 1



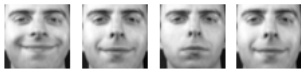
Cluster 22

17 17 17



Cluster 23

18 18 18 18



Cluster 24

22 22



Cluster 25

32 32 32



Cluster 26

13 13 13 13 13



Cluster 27

9 9



Cluster 28

30 30 30 30



Cluster 29